

**UNIVERSIDADE FEDERAL DE MINAS GERAIS
ESCOLA DE ENGENHARIA
CURSO DE ENGENHARIA DE SISTEMAS**

MONITORAMENTO E OTIMIZAÇÃO DO CONSUMO DE RECURSOS EM APLICATIVOS FLUTTER

Proposta de Ferramenta de Apoio ao Desenvolvimento

Beatriz Vocurca Frade

Orientador: Prof. Ramon Lacerda Marques

Data da defesa: 18/06/2026, às 15h

Belo Horizonte
2026

Resumo

O monitoramento de desempenho em aplicações móveis depende, na prática, de ferramentas externas que exigem configuração especializada e interpretação manual de dados, uma barreira que restringe sua adoção ao longo do desenvolvimento cotidiano. Este trabalho apresenta o `collector_flutter`, um pacote Dart/Flutter de código aberto que incorpora coleta de métricas, análise heurística e visualização em tempo real diretamente na aplicação monitorada, sem dependências nativas adicionais para as métricas Dart e com *platform channels* para CPU e bateria em Android.

O pacote adota arquitetura de três camadas (*Data*, *Domain* e *Presentation*) baseada nos princípios da *Clean Architecture*, coletando métricas de renderização (FPS, *jank*, percentis P50/P95/P99), uso de memória RSS e *heap*, tráfego HTTP, uso de CPU por processo, nível de bateria e eventos customizados do desenvolvedor. Um motor heurístico analisa essas métricas a cada ciclo de coleta e gera recomendações de otimização classificadas por severidade, exibidas em um painel visual integrado. A exportação da sessão em JSON e CSV é disponibilizada via `ExportService`.

O desenvolvimento seguiu um ciclo de vida iterativo estruturado em quatro fases: revisão bibliográfica, especificação de requisitos, implementação do pacote e avaliação experimental. A validação funcional em quatro cenários controlados (*rebuilds* intensivos, carga de rede, alocação de memória e carga combinada) indicou que o sistema permanece operável e que as heurísticas respondem aos sintomas induzidos. A suíte automatizada é composta por 60 casos de teste distribuídos em quatro arquivos, cobrindo os módulos de análise, recomendação, exportação e os *data sources* de CPU e bateria.

Palavras-chave: Flutter. Monitoramento de desempenho. Profiling. Coleta de métricas em tempo real. Eficiência energética. Sustentabilidade digital. Clean Architecture.

Sumário

Resumo	i
Lista de Figuras	v
1 Introdução	1
1.1 Propósito	1
1.2 Público-Alvo	2
1.3 Motivação e Justificativa	2
1.4 Objetivos	3
1.5 Visão Geral do Documento	3
1.6 Revisão Bibliográfica	4
2 Contexto Social, Ambiental e Ético	7
2.1 Impactos Socioeconômicos	7
2.2 Impactos Sociais e Inclusão Digital	8
2.3 Impactos Ambientais e Eficiência Computacional	8
2.4 Aspectos Éticos no Monitoramento de Aplicações	9
2.5 Responsabilidade Tecnológica no Desenvolvimento de Software	9
2.6 Síntese	9
3 Metodologia e Desenvolvimento	11
3.1 Abordagem Metodológica	11
3.1.1 Natureza da Pesquisa	11
3.1.2 Objetivos da Abordagem	11
3.1.3 Justificativa da Escolha Metodológica	12
3.1.4 Ciclo de Vida da Pesquisa e do Desenvolvimento	12
3.1.5 Fase 2 – Levantamento de Requisitos e Especificação	15
3.1.6 Fase 3 – Projeto e Implementação do Pacote	16
3.1.7 Fase 4 – Avaliação Experimental e Validação	17
3.2 Descrição do Pacote e Arquitetura da Solução	19
3.2.1 Concepção e Premissas de Projeto	19
3.2.2 Arquitetura Geral do Sistema	20
3.2.3 Componentes Principais	20
3.2.4 Interface e Painel de Visualização	21
3.2.5 Regras de Recomendação	25
3.2.6 Demonstração Prática e Disponibilização do Código-Fonte	25
3.3 Critérios de Avaliação	26

3.3.1	Coerência das Métricas Coletadas	26
3.3.2	Overhead Introduzido pelo Sistema	26
3.3.3	Eficácia das Recomendações Geradas	26
3.3.4	Clareza da Interface	27
3.4	Ferramentas e Ambientes de Desenvolvimento	27
3.4.1	Ambientes de Programação e IDEs	27
3.4.2	SDKs e Dependências Utilizadas	27
3.4.3	Ambientes de Teste e Dispositivos	28
3.4.4	Ferramentas de Referência e Validação	28
3.4.5	Frameworks e Bibliotecas de Apoio	28
3.4.6	Ambientes de Testes Automatizados	29
3.5	Procedimentos de Teste e Coleta de Dados	30
3.5.1	Procedimento de Execução dos Testes	30
3.5.2	Definição dos Cenários Experimentais	30
3.5.3	Coleta de Métricas	31
3.5.4	Métodos de Comparação	32
3.5.5	Análise dos Resultados	32
3.5.6	Controle Experimental e Reprodutibilidade	32
4	Resultados e Discussão	34
4.1	Desenvolvimento do Pacote	34
4.1.1	Módulos Implementados	34
4.1.2	Estrutura da Biblioteca	35
4.2	Validação da Arquitetura	36
4.2.1	Conformidade Arquitetural	36
4.2.2	Comportamento Observado em Execução	36
4.3	Análise dos Cenários Experimentais	37
4.3.1	Cenário 1: Reconstruções Intensivas de Interface	38
4.3.2	Cenário 2: Chamadas de Rede Simultâneas	38
4.3.3	Cenário 3: Alocação Intensiva de Memória	39
4.3.4	Cenário 4: Carga Combinada	39
4.3.5	Comparação Qualitativa com ADB	39
4.3.6	Evidências Complementares da Validação	40
4.4	Evolução do Pacote: Limitações Superadas	41
4.5	Resumo do Capítulo	41
5	Conclusões	42
5.1	Considerações Finais	42
5.2	Contribuições do Trabalho	43
5.3	Limitações do Estudo	43
5.4	Trabalhos Futuros	44
A	Evidências Visuais e Leituras ADB	45

B	Casos de Teste Automatizados	51
B.1	Análise e Estatísticas	51
B.2	CPU e Bateria	52
B.3	Exportação	53
B.4	Recomendações	53

Lista de Figuras

3.1	Ciclo de vida metodológico adotado (iterativo e incremental).	13
3.2	Estratégia metodológica da pesquisa, com foco em revisão bibliográfica, requisitos, implementação e validação funcional do pacote.	13
3.3	Arquitetura local e fluxo de dados do pacote <code>collector_flutter</code> : coleta, análise, recomendação e apresentação no painel integrado.	16
3.4	Painel do aplicativo em condição de desempenho estável, com indicadores verdes e recomendações informativas.	23
3.5	Painel do aplicativo com anomalias detectadas, como FPS baixo e uso de memória elevado.	24
3.6	Ícone do aplicativo de demonstração <code>Collector Flutter</code>	24
4.1	Evidências visuais do painel durante a validação: à esquerda, condição estável; à direita, condição com gargalos detectados, eventos de rede e alertas ativos.	38
A.1	Evidência visual do cenário de reconstruções intensivas de interface.	46
A.2	Evidência visual do cenário de chamadas de rede simultâneas.	47
A.3	Evidência visual do cenário de alocação intensiva de memória.	48
A.4	Evidência visual do alerta de <i>jank</i> no painel.	49
A.5	Evidência visual do cenário de carga combinada.	50

Capítulo 1

Introdução

O Flutter consolidou-se como um *framework* aberto para desenvolvimento de aplicações multiplataforma a partir de uma única base de código, com suporte a dispositivos móveis, Web, desktop e sistemas embarcados [10]. Essa adoção amplia a produtividade das equipes, mas também impõe desafios específicos à cadeia de desenvolvimento: aplicações móveis modernas combinam interfaces reativas, animações, chamadas de rede, gerenciamento de estado e execução em dispositivos com capacidades muito diferentes.

Embora ferramentas como *Flutter DevTools*, *Android Profiler* e *Perfetto* sejam tecnicamente maduras, sua arquitetura externa ao fluxo de desenvolvimento cria barreiras que dificultam a adoção contínua. As principais barreiras identificadas são:

- Requerem saída do ambiente de desenvolvimento (aplicações separadas, WebSocket, conexão USB);
- Demandam configuração manual e interpretação de dados complexos;
- Não fornecem recomendações automáticas de otimização;
- Impõem custo cognitivo elevado em ciclos iterativos rápidos.

Na prática, essas barreiras levam equipes a postergar o diagnóstico de desempenho para fases tardias do projeto, quando regressões já podem ter impactado o usuário final.

Este trabalho propõe o `collector_flutter`: um pacote Dart de código aberto que integra coleta de métricas, análise heurística e visualização diretamente na aplicação monitorada. O diferencial reside na **acessibilidade**: basta adicionar o pacote e realizar três chamadas de código para ativar o painel de diagnóstico, que opera como uma tela comum do aplicativo, sem configurações externas.

1.1 Propósito

O propósito deste trabalho é projetar, implementar e validar funcionalmente o `collector_flutter`, uma biblioteca Dart para monitoramento de desempenho em tempo real em aplicações Flutter. O pacote coleta métricas de renderização (FPS, *jank*, percentis de tempo de quadro), uso de memória RSS e *heap*, tráfego HTTP e eventos customizados definidos pelo desenvolvedor. Um motor heurístico analisa essas métricas a cada ciclo de coleta e exibe recomendações de otimização classificadas por severidade em um painel visual integrado.

A proposta não substitui ferramentas de diagnóstico especializadas como o *Flutter DevTools* ou o *Android Profiler*. Seu papel é diferente: funcionar como um termômetro contínuo durante o desenvolvimento, capaz de sinalizar regressões de desempenho no mesmo momento em que o código é exercitado, sem exigir que o desenvolvedor alterne de contexto para uma ferramenta externa.

1.2 Público-Alvo

O trabalho se destina a três grupos principais:

- Desenvolvedores Flutter que precisam de feedback de desempenho integrado ao fluxo de desenvolvimento, especialmente em equipes pequenas sem recursos para ciclos de *profiling* dedicados;
- Pesquisadores de engenharia de software interessados no problema de instrumentação não-intrusiva em frameworks reativos, em especial na relação entre frequência de coleta, precisão das métricas e *overhead* introduzido;
- Estudantes de computação e sistemas que buscam um exemplo concreto de como aplicar princípios de *Clean Architecture* e programação reativa em Dart para construir uma biblioteca com separação clara entre coleta, análise e apresentação de dados.

1.3 Motivação e Justificativa

A relação entre desempenho e consumo energético em aplicações móveis é documentada pela literatura desde o trabalho seminal de Pathak *et al.* [1], que demonstrou a importância de modelos finos de energia para compreender o custo de operações realizadas por aplicações em smartphones. Estudos posteriores mostram que métricas como tempo de execução, uso de memória, chamadas de rede e padrões de uso de APIs ajudam a explicar variações de consumo energético em aplicações móveis reais [3, 6, 8].

Esses dados têm implicações práticas diretas: um aplicativo com renderização ineficiente tende a consumir mais recursos, responder com menor fluidez e funcionar pior em aparelhos com restrições de processamento e memória. A sustentabilidade de um ecossistema digital passa, portanto, pela capacidade dos desenvolvedores de identificar e corrigir ineficiências antes que estas cheguem ao usuário final.

O problema é que a detecção precoce de regressões de desempenho requer instrumentação contínua, e as ferramentas disponíveis tendem a operar fora do fluxo direto da aplicação. O *Flutter DevTools* fornece recursos para inspeção de interface, análise de memória, *timeline* de quadros, perfil de CPU e rede [11], mas exige que o desenvolvedor acesse uma ferramenta separada e interprete os sinais coletados. Essa lacuna, entre a disponibilidade dos dados e a capacidade de agir sobre eles no momento certo, é o problema central que este trabalho endereça.

1.4 Objetivos

O objetivo geral deste trabalho é desenvolver e validar funcionalmente uma biblioteca Flutter para monitoramento de desempenho embutido. Essa biblioteca recebeu o nome de `collector_flutter`. Busca-se demonstrar que é possível construir um sistema integrado ao aplicativo, capaz de coletar métricas, analisá-las por heurísticas e apresentar recomendações acionáveis durante o desenvolvimento.

Os objetivos específicos são:

1. Realizar revisão bibliográfica estruturada sobre monitoramento de recursos, eficiência energética e ferramentas de *profiling* em aplicações móveis, com foco no ecossistema Flutter.
2. Identificar e documentar as limitações das ferramentas de diagnóstico existentes quanto à integração ao fluxo de desenvolvimento e à geração de recomendações automáticas.
3. Projetar uma arquitetura modular de três camadas que separe coleta, análise e apresentação de métricas, garantindo baixo acoplamento e extensibilidade.
4. Implementar os módulos de coleta (FPS, memória, rede, eventos), análise heurística e visualização em tempo real como um pacote Dart publicável.
5. Validar o pacote em quatro cenários experimentais controlados, confirmando a operacionalidade do sistema e a pertinência das heurísticas implementadas.
6. Verificar qualitativamente o impacto percebido do monitoramento e a coerência das métricas coletadas com apoio de ferramentas de referência, especialmente leituras complementares obtidas por `adb shell dumpsys gfxinfo` e `adb shell dumpsys meminfo`.
7. Disponibilizar o pacote como código aberto, com documentação técnica suficiente para reprodução e extensão por outros pesquisadores.

1.5 Visão Geral do Documento

O Capítulo 2 apresenta a fundamentação teórica, reunindo a revisão bibliográfica sobre monitoramento de recursos e ferramentas de diagnóstico existentes, e o contexto de humanidades que situa o trabalho nas dimensões de sustentabilidade digital, impacto social e responsabilidade ética.

O Capítulo 3 descreve a metodologia adotada, incluindo a natureza da pesquisa, o ciclo de vida iterativo, a arquitetura do pacote, os requisitos funcionais e não funcionais, a especificação dos módulos implementados e os procedimentos de validação funcional.

O Capítulo 4 apresenta os resultados do desenvolvimento: o estado de implementação de cada módulo, a evolução das limitações identificadas, a validação arquitetural e as observações qualitativas dos quatro cenários experimentais.

O Capítulo 5 consolida as conclusões do trabalho, apresenta as contribuições alcançadas e discute as limitações metodológicas do estudo.

1.6 Revisão Bibliográfica

A construção de uma ferramenta de monitoramento de desempenho para Flutter requer embasamento em três eixos complementares: (i) a natureza das métricas de execução em plataformas móveis e as técnicas para capturá-las; (ii) as soluções existentes e suas limitações; e (iii) as abordagens acadêmicas que investigam o problema de instrumentação não-intrusiva e eficiência energética. Esta seção sintetiza criticamente esses eixos e identifica a lacuna que este trabalho busca preencher.

Métricas de Desempenho e Técnicas de Coleta em Aplicações Móveis

O estudo de métricas de desempenho em dispositivos móveis consolidou-se com o trabalho de Pathak *et al.* [1], que demonstrou a importância de modelos finos de energia para compreender como chamadas de sistema e uso de componentes afetam o consumo em smartphones. Revisões posteriores, como a de Hoque *et al.* [5], organizam esse campo ao comparar técnicas de modelagem, *profiling* e depuração energética em dispositivos móveis.

No contexto de estimativa de energia, Hao *et al.* [2] propuseram uma abordagem baseada em análise de programa para estimar consumo em nível de código, enquanto Li *et al.* [3] analisaram aplicações Android reais e apontaram a relevância de chamadas de rede, APIs do sistema e estruturas repetitivas no consumo observado. Esses trabalhos sustentam a escolha de métricas observáveis, como tempo de quadro, memória e rede, para inferir gargalos de execução.

Pesquisas recentes reforçam que desempenho móvel deve ser analisado de forma multidimensional. Rua e Saraiva [6] investigaram energia, tempo de execução e memória em larga escala, mostrando que práticas de código e mudanças funcionais podem afetar indicadores de desempenho de formas distintas. Em paralelo, Nawrocki *et al.* [9] compararam abordagens nativas e multiplataforma, evidenciando que a escolha do *framework* influencia métricas como responsividade, memória e custo de execução.

Estudos com foco em monitoramento automatizado seguem uma linha complementar. Ravindranath *et al.* [4] propuseram o AppInsight, um sistema para monitoramento de desempenho de aplicações móveis em campo, com identificação de caminhos críticos em transações assíncronas. O trabalho mostra a relevância de coletar dados próximos ao uso real da aplicação e de apresentar informações que ajudem desenvolvedores a localizar gargalos.

Do ponto de vista de análise estática e uso de APIs, Bangash *et al.* [8] demonstram que é possível estimar consumo energético associado ao uso de APIs em aplicativos móveis sem depender exclusivamente de medições físicas repetidas. Essa direção fundamenta a escolha do `collector_flutter` por combinar métricas dinâmicas e recomendações baseadas em padrões de implementação.

Ferramentas de Monitoramento Existentes

O ecossistema de desenvolvimento móvel dispõe de ferramentas maduras de *profiling*, mas com características que limitam sua adoção contínua durante o desenvolvimento. A Tabela 1.1 apresenta um comparativo estruturado entre as principais soluções disponíveis.

A análise da tabela revela um padrão consistente: ferramentas externas como o DevTools e o Android Profiler fornecem dados de alta qualidade, mas exigem saída do ambiente de

Tabela 1.1: Comparativo de ferramentas de monitoramento e análise de desempenho.

Ferramenta	Principais Funcionalidades	Limitações Relevantes
Ferramentas e Soluções de Monitoramento		
Flutter DevTools [11]	Inspeção de widgets, análise de memória, <i>timeline</i> de frames e perfil de CPU.	Opera como aplicação separada; requer conexão via WebSocket ao processo; ausência de recomendações automáticas; dependência do modo <i>debug/profile</i> .
Perfetto (AOSP) [13]	Rastreamento detalhado de processos e eventos do sistema Android com suporte a múltiplas camadas.	Configuração manual de sessões de rastreamento; interpretação de traços binários; alto custo cognitivo; não integrado ao fluxo de desenvolvimento.
Android Profiler [12]	Monitoramento em tempo real de CPU, memória, rede e energia diretamente no Android Studio.	Restrito a Android; requer conexão USB em modo <i>profile</i> ; integração limitada com frameworks multiplataforma.
Xcode Instruments [14]	Coleta avançada de métricas de energia, CPU e renderização para dispositivos iOS.	Exclusivo ao ecossistema Apple; coleta manual de dados; elevado <i>overhead</i> em algumas configurações.
Firebase Performance [15]	Monitoramento automático de tempos de carregamento e chamadas de rede em produção.	Granularidade limitada; sem métricas de renderização ou CPU; voltado a produção, não ao desenvolvimento.

desenvolvimento, configuração explícita e interpretação manual dos resultados. A literatura sobre *profiling* móvel também aponta que a coleta de dados de desempenho é dificultada por execução assíncrona, múltiplas camadas de sistema e limitações próprias dos dispositivos móveis [4, 5].

Esse cenário aponta para uma oportunidade de contribuição: sistemas que não apenas coletem dados, mas também interpretem sinais recorrentes e comuniquem ações possíveis. Para o caso de aplicações Flutter, essa interpretação pode ser feita a partir de sintomas observáveis como aumento de tempo de quadro, *jank*, crescimento de memória e frequência de reconstruções.

Estudos Acadêmicos Relacionados

A pesquisa em monitoramento de desempenho para aplicações móveis indica que não basta observar uma única métrica isolada. Estudos empíricos recentes combinam tempo de execução, energia, memória e características do código para compreender o comportamento de aplicações em diferentes cenários [6, 7]. Essa visão orienta o `collector_flutter` a combinar métricas de renderização, memória, rede e eventos customizados.

O AppInsight [4] demonstra a relevância de monitorar aplicações em execução e aproximar a análise de desempenho do comportamento real observado. Já os estudos de energia baseados em análise de programa e uso de APIs [2, 8] indicam que recomendações automatizadas podem se apoiar em sinais estruturais do código e em métricas de execução.

Em perspectiva mais ampla, revisões sobre energia em aplicações Android mostram que diagnóstico energético envolve métodos de estimativa, componentes de hardware, tipos de defeitos e abordagens de análise de programa [7]. O motor heurístico do `collector_flutter` responde a essa lacuna de forma pragmática: em vez de tentar substituir ferramentas profundas, ele traduz sintomas frequentes em recomendações acionáveis durante o desenvolvimento.

Lacuna de Pesquisa

A revisão evidencia que as ferramentas de *profiling* disponíveis para Flutter são tecnicamente maduras, mas que sua arquitetura externa ao processo de execução cria um atrito de uso que limita sua adoção contínua. As abordagens acadêmicas investigam métricas, técnicas de coleta e impacto energético, mas raramente propõem sistemas completos, da coleta à recomendação, que operem de forma embutida e com *overhead* explicitamente controlado.

O presente trabalho propõe o `collector_flutter` como contribuição a essa lacuna: uma biblioteca que integra coleta automática de métricas de renderização, memória, rede, CPU, bateria e eventos customizados, análise heurística e geração de recomendações em um único componente. A hipótese central é que esse conjunto de características produz uma ferramenta com menor atrito de adoção em comparação às alternativas externas existentes, o que foi verificado por meio de validação funcional e qualitativa em cenários controlados.

Capítulo 2

Contexto Social, Ambiental e Ético

2.1 Impactos Socioeconômicos

Eficiência computacional tem valor econômico mensurável. Em infraestrutura de servidores, a relação é direta: ciclos de CPU poupados são custos de nuvem reduzidos. Em dispositivos móveis, o mecanismo é diferente mas igualmente concreto: aplicações mais leves tendem a consumir menos bateria, exigir menos memória e funcionar melhor em dispositivos de menor capacidade [5, 6]. Em contextos de menor poder aquisitivo, a longevidade do hardware é economicamente relevante para o usuário final.

Do ponto de vista social, aplicações mal otimizadas reforçam desigualdades de acesso. Um serviço de saúde ou de educação que funciona bem em um Pixel 8 mas gera *jank* constante em um dispositivo de entrada cria uma experiência de segunda classe para usuários que já possuem menor capacidade de escolha. Otimizar desempenho, nesse contexto, também é uma questão de equidade no acesso a serviços digitais.

Ferramentas de software raramente são neutras em seus efeitos. Uma biblioteca de monitoramento de desempenho para Flutter parece, à primeira vista, um artefato técnico restrito ao ecossistema de desenvolvimento. Ainda assim, ela carrega decisões de projeto com implicações que vão além da eficiência computacional: para quem a ferramenta é acessível, quais dados ela coleta, como esses dados são tratados e em que medida ela apoia práticas de desenvolvimento mais conscientes do impacto ambiental e social das aplicações produzidas. Este capítulo situa o `collector_flutter` nessas dimensões mais amplas.

Sustentabilidade Digital e o Custo Energético do Software

A relação entre eficiência de software e consumo energético ganhou escala com a proliferação de dispositivos móveis. Revisões sobre energia em aplicações Android mostram que defeitos energéticos podem surgir de diferentes fontes, como uso de rede, sensores, chamadas de API, processamento em segundo plano e decisões de arquitetura [7]. Estudos empíricos em larga escala também indicam que energia, tempo de execução e memória nem sempre variam da mesma forma, o que exige monitoramento contínuo e interpretação cuidadosa das métricas [6].

Essa literatura tem uma implicação prática específica para o desenvolvimento de software: ineficiências locais se multiplicam pela escala de uso. Um *rebuild* desnecessário, uma requisição repetida ou uma alocação de memória evitável podem parecer pequenos durante

um teste isolado, mas tornam-se relevantes quando repetidos ao longo de muitas sessões e dispositivos. Quando somado, esse efeito se torna ambientalmente e economicamente relevante.

O `collector_flutter` insere-se nessa perspectiva ao tornar visível, para o desenvolvedor, o custo computacional das escolhas de implementação. Ao identificar *jank* recorrente, tendências de crescimento de memória ou volume excessivo de requisições de rede, a ferramenta ajuda a melhorar a experiência do usuário e a reduzir o consumo energético da aplicação resultante.

2.2 Impactos Sociais e Inclusão Digital

Desempenho não é uma preocupação uniforme ao longo da pirâmide socioeconômica. Aplicações que funcionam bem em dispositivos de alta capacidade, como Pixels, iPhones e Galaxy S, podem degradar de forma severa em aparelhos de entrada. Essa assimetria tem consequências concretas: serviços de saúde, educação e comunicação que se tornam praticamente inutilizáveis em dispositivos de menor custo criam uma experiência de segunda classe para usuários que já possuem menor poder de escolha.

A otimização de desempenho em aplicações Flutter é, portanto, também uma questão de equidade de acesso. Uma aplicação que consome menos CPU, memória e rede tende a operar adequadamente em uma gama mais ampla de dispositivos. Ferramentas que facilitam a identificação e correção de ineficiências durante o desenvolvimento favorecem essa inclusão ao reduzir a distância entre uma aplicação funcionalmente correta e uma aplicação acessível para todos os usuários potenciais.

O `collector_flutter` foi projetado para operar sem restrições de plataforma ou dispositivo. A coleta de métricas via `SchedulerBinding` e `VM Service` funciona igualmente em aparelhos de alta e baixa capacidade; as heurísticas de recomendação são calibradas para detectar comportamentos que afetam desempenho em dispositivos com restrições de hardware. Isso não torna a ferramenta um instrumento de inclusão digital por si só, mas alinha suas escolhas de projeto com esse valor.

2.3 Impactos Ambientais e Eficiência Computacional

O ciclo de vida de um dispositivo móvel tem dois determinantes principais: a durabilidade física do hardware e a compatibilidade do software disponível com suas especificações. Aplicações cada vez mais exigentes aceleram a obsolescência percebida dos dispositivos. Quando um smartphone de três anos não consegue executar as versões mais recentes dos aplicativos principais sem *jank* severo, o incentivo para descartá-lo aumenta, mesmo que o hardware esteja fisicamente íntegro.

Resíduos eletrônicos constituem uma das correntes de lixo de crescimento mais rápido no mundo, conforme sintetizado pelo *Global E-waste Monitor* [16]. Uma contribuição modesta, mas concreta, da engenharia de software para mitigar esse problema é o desenvolvimento de aplicações que continuem funcionando adequadamente em dispositivos mais antigos. Ferramentas de monitoramento que tornam visível o custo computacional do software e geram recomendações práticas para reduzi-lo apoiam esse objetivo.

2.4 Aspectos Éticos no Monitoramento de Aplicações

Sistemas de monitoramento de desempenho coletam informações sobre o comportamento de aplicações em execução. Em contextos de produção, onde o monitoramento ocorre sobre dispositivos de usuários reais, essa coleta levanta questões éticas legítimas: quais dados são coletados, por quanto tempo, por quem podem ser acessados e para quais finalidades podem ser utilizados.

O `collector_flutter` foi projetado com o princípio de minimização de dados: ele coleta exclusivamente métricas de desempenho do sistema, como tempo de quadro, uso de memória, contagem de requisições e eventos definidos pelo desenvolvedor. Nenhum dado de identificação de usuário, conteúdo de comunicação ou informação pessoal é registrado. As métricas existem apenas em memória durante a sessão de uso da ferramenta e, na versão atual, não são transmitidas a servidores externos.

Essa delimitação é uma escolha ética e também técnica. Um sistema de monitoramento que coleta mais dados do que o necessário para seus objetivos de diagnóstico introduz riscos de privacidade desnecessários e aumenta o *overhead* sem benefício correspondente para o desenvolvedor. O princípio de minimização de dados serve, portanto, tanto à ética quanto à eficiência.

A disponibilização do código-fonte como software aberto contribui adicionalmente para a transparência: qualquer desenvolvedor pode auditar o que é e o que não é coletado, verificar as heurísticas de recomendação e propor melhorias. Essa abertura é coerente com os padrões de reprodutibilidade científica recomendados por Wohlin *et al.* [20] para pesquisas experimentais em engenharia de software.

2.5 Responsabilidade Tecnológica no Desenvolvimento de Software

A noção de responsabilidade tecnológica, entendida como a corresponsabilidade de desenvolvedores e pesquisadores pelos impactos das soluções que produzem, é cada vez mais central na engenharia de software contemporânea. Na prática, isso significa considerar a correção funcional e o desempenho técnico de um sistema junto com suas implicações de acessibilidade, consumo de recursos, privacidade e longevidade.

Este trabalho é uma contribuição técnica: um pacote Dart com arquitetura bem definida e avaliação experimental em cenários controlados. Ele também expressa uma escolha de projeto: o diagnóstico de desempenho deve ser acessível ao desenvolvedor que trabalha em uma equipe pequena, sem orçamento para ciclos de *profiling* dedicados. A eficiência computacional deve ser monitorada continuamente, e ferramentas que democratizam esse diagnóstico ajudam a construir um ecossistema digital mais eficiente e mais equitativo.

2.6 Síntese

O monitoramento de desempenho em aplicações Flutter, enquanto problema técnico, é bem delimitado: coletar métricas, analisá-las e comunicar resultados ao desenvolvedor. Enquanto problema de engenharia com dimensões sociais e ambientais, é mais amplo: tornar esse di-

agnóstico acessível reduz o consumo energético das aplicações produzidas, amplia sua compatibilidade com dispositivos de menor capacidade e incentiva práticas de desenvolvimento mais conscientes de seu impacto. O `collector_flutter` é uma resposta ao primeiro problema e busca apoiar, de forma instrumental, o segundo.

Capítulo 3

Metodologia e Desenvolvimento

3.1 Abordagem Metodológica

3.1.1 Natureza da Pesquisa

A presente pesquisa caracteriza-se como um estudo de natureza **aplicada**, pois visa desenvolver e validar funcionalmente um pacote de monitoramento de desempenho para aplicações Flutter. Segundo Gil [17], pesquisas aplicadas têm como propósito “gerar conhecimentos voltados à aplicação prática, dirigidos à solução de problemas específicos”.

Nesse contexto, o `collector_flutter` oferece uma solução técnica para coletar, analisar e interpretar métricas de execução em tempo real, apoiando o aprimoramento da eficiência e da sustentabilidade digital de aplicativos móveis.

A pesquisa também apresenta abordagem **experimental**, uma vez que envolve a manipulação controlada de variáveis e a observação empírica dos resultados em ambientes reais. Conforme Lakatos e Marconi [18], o método experimental é adequado quando se pretende “testar hipóteses mediante o controle e a observação dos efeitos provocados por determinadas condições”.

3.1.2 Objetivos da Abordagem

A adoção de uma abordagem aplicada e experimental permite atender a objetivos técnicos e científicos específicos, entre os quais se destacam:

- **Demonstrar a viabilidade técnica** de um sistema de monitoramento autônomo e totalmente embutido no ecossistema Flutter, sem dependência de ferramentas nativas externas (Android Developers [12]);
- **Observar funcionalmente o impacto da coleta de métricas** sobre o desempenho da aplicação, verificando se o monitoramento permanece ativo sem comprometer a interação com o aplicativo de demonstração;
- **Verificar a coerência dos dados coletados**, confrontando-os qualitativamente com sinais observados no painel e com leituras complementares obtidas por ferramentas de referência, em especial comandos ADB documentados nos resultados [12];

- **Validar qualitativamente as recomendações heurísticas**, verificando se os alertas gerados são compatíveis com os gargalos induzidos nos cenários de teste;
- **Promover a reprodutibilidade científica**, disponibilizando publicamente o código-fonte e a documentação técnica do pacote, conforme recomendam Wohlin *et al.* [20].

Esses objetivos orientam tanto o delineamento experimental quanto as etapas de desenvolvimento do sistema, assegurando coerência entre a fundamentação teórica, o projeto técnico e a avaliação empírica dos resultados.

3.1.3 Justificativa da Escolha Metodológica

A escolha de uma metodologia de natureza aplicada e abordagem experimental fundamenta-se na necessidade de validar empiricamente o comportamento de sistemas de monitoramento em contextos reais de execução. Conforme destaca Yin [19], abordagens experimentais e estudos de caso são apropriados quando o pesquisador busca compreender relações de causa e efeito entre variáveis observáveis em ambientes complexos como é o caso de aplicações móveis em execução sobre diferentes plataformas e dispositivos.

No caso do `collector_flutter`, a observação direta de variáveis como *frame time*, reconstruções de widgets e uso de memória é indispensável para analisar se os sinais exibidos pelo painel acompanham os cenários executados. Estudos empíricos recentes sobre desempenho móvel reforçam que tempo de execução, memória e energia devem ser avaliados de forma conjunta, pois cada indicador pode reagir de modo distinto às mudanças realizadas no software [6]. Por isso, a análise experimental é essencial para associar métricas observáveis a efeitos percebidos durante a execução do aplicativo.

Assim, a abordagem metodológica proposta combina:

- fundamentos teóricos sobre desempenho, eficiência e sustentabilidade digital;
- execução controlada do pacote em ambiente Android;
- análise qualitativa dos resultados observados com apoio de ferramentas nativas de referência.

Essa integração entre teoria e prática sustenta a avaliação do sistema proposto e permite que ele seja reproduzido e aprimorado em diferentes contextos de desenvolvimento.

3.1.4 Ciclo de Vida da Pesquisa e do Desenvolvimento

O processo adotado neste trabalho segue um **ciclo de vida iterativo e incremental**, característico de projetos de engenharia de software com validação empírica, combinando desenvolvimento de pacote e experimentação controlada. Esse ciclo foi estruturado com base em princípios de engenharia de software (Pressman e Maxim [21]; Sommerville [22]) e em diretrizes de experimentação em engenharia de software (Wohlin *et al.* [20]), garantindo rastreabilidade entre objetivos, decisões de projeto e resultados.

A Figura 3.1 representa o ciclo metodológico adotado, organizado em fases com **entradas, atividades, artefatos gerados e critérios de término**. O ciclo contempla dois momentos do trabalho: (i) **TCC 1**, voltado à fundamentação e ao protótipo inicial; e (ii) **TCC 2**, voltado à consolidação do pacote, aos testes funcionais e à análise qualitativa dos resultados.

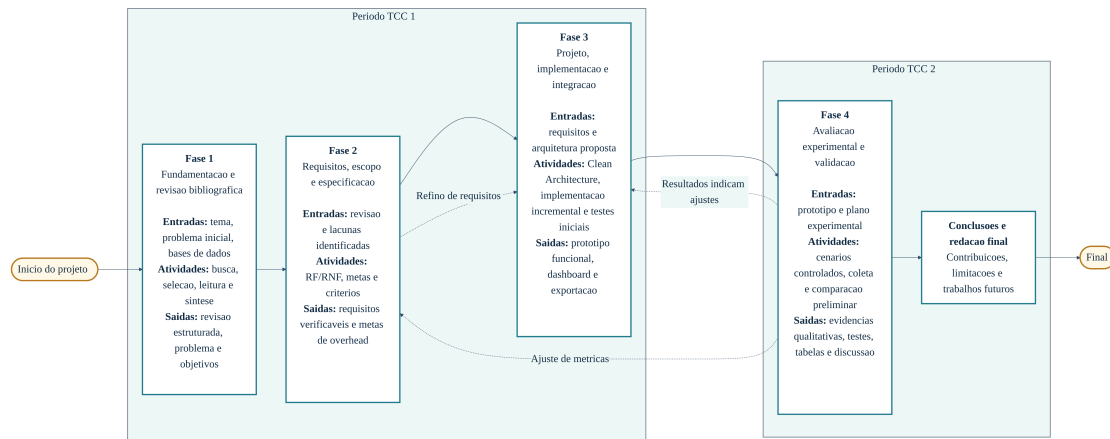


Figura 3.1: Ciclo de vida metodológico adotado (iterativo e incremental).

A Figura 3.2 sintetiza a estratégia metodológica adotada no trabalho, articulando fundamentação teórica, levantamento de requisitos, construção do pacote e validação funcional. Essa organização evidencia que a avaliação realizada tem caráter qualitativo e funcional, sem análise estatística formal ou estudo externo de usabilidade.

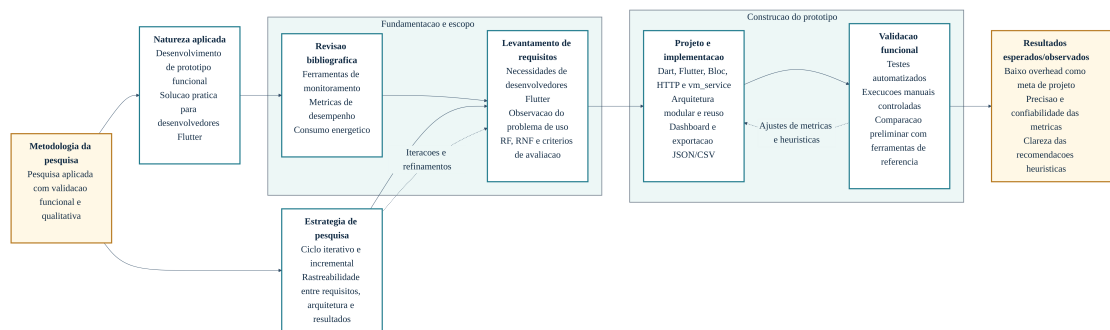


Figura 3.2: Estratégia metodológica da pesquisa, com foco em revisão bibliográfica, requisitos, implementação e validação funcional do pacote.

Visão Geral das Fases e Entregáveis

O ciclo de vida foi operacionalizado em quatro fases principais, descritas a seguir. Para cada fase, definem-se claramente: (i) o objetivo, (ii) as atividades, (iii) os artefatos produzidos e (iv) o critério de conclusão.

Fase 1: Fundamentação e Revisão Bibliográfica **Objetivo:** construir o embasamento teórico sobre desempenho, profiling, métricas e eficiência energética em aplicações móveis, delimitando o problema e justificando a solução proposta. **Atividades:** busca sistemática por palavras-chave, leitura e seleção de estudos, organização temática e consolidação de lacunas na literatura. **Artefatos/saídas:** (i) revisão bibliográfica estruturada, (ii) definição do problema, (iii) objetivos e expectativas de validação. **Critério de conclusão:** existência de um referencial teórico suficiente para sustentar as métricas monitoradas e as escolhas de projeto do pacote.

Fase 2: Requisitos, Escopo e Especificação **Objetivo:** transformar o problema de pesquisa em requisitos verificáveis e critérios de sucesso, descrevendo claramente o que o pacote deve fazer e quais limites deve respeitar. **Atividades:** elicitação e análise de requisitos, definição de critérios de qualidade e operacionalização das métricas (o que medir e como medir). **Artefatos/saídas:** (i) requisitos funcionais e não funcionais, (ii) critérios de avaliação, (iii) definição de metas (ex.: limite de *overhead*). **Critério de conclusão:** requisitos claros e mensuráveis, capazes de orientar implementação e avaliação experimental.

Fase 3: Projeto, Implementação e Integração do Pacote **Objetivo:** projetar e implementar o `collector_flutter` de forma modular e testável, com integração a aplicações Flutter e capacidade de coleta/visualização local. **Atividades:** definição da arquitetura (camadas e módulos), implementação incremental das funcionalidades, integração entre módulos e testes iniciais. **Artefatos/saídas:** (i) pacote funcional, (ii) arquitetura documentada (diagramas e componentes), (iii) painel (*dashboard*) integrado, (iv) registro e exportação de métricas. **Critério de conclusão:** pacote executável, capaz de coletar e exibir métricas em tempo real em ambiente controlado.

Fase 4: Avaliação Experimental, Análise e Validação **Objetivo:** validar funcionalmente o pacote, observando se as métricas coletadas e as recomendações geradas são coerentes com os gargalos induzidos em cenários controlados. **Atividades:** definição dos cenários experimentais, execução manual controlada, coleta automatizada de telemetria, inspeção dos resultados e confronto qualitativo com sinais obtidos por ADB. **Artefatos/saídas:** (i) registros de telemetria, (ii) tabela de cenários, (iii) evidências visuais do painel, (iv) leituras complementares via ADB, (v) discussão de resultados e limitações. **Critério de conclusão:** evidências suficientes para demonstrar a operacionalidade do pacote e a pertinência das heurísticas implementadas.

Iterações e Rastreabilidade

Embora as fases sejam apresentadas de forma sequencial, o ciclo adotado é **iterativo**: resultados parciais de testes e integração podem motivar ajustes em requisitos, arquitetura ou heurísticas, desde que tais mudanças sejam registradas e justificadas. Para garantir rastreabilidade, cada funcionalidade do pacote foi associada a: (i) um requisito (RF/RNF), (ii) um módulo da arquitetura e (iii) um critério de avaliação experimental, permitindo ligação direta entre o que foi proposto, implementado e validado.

Distribuição do ciclo de desenvolvimento

O ciclo metodológico foi distribuído em dois macroperíodos:

- **TCC 1:** foco nas Fases 1, 2 e início da Fase 3, resultando em fundamentação teórica consolidada e protótipo inicial funcional.
- **TCC 2:** foco na conclusão da Fase 3 e execução da Fase 4 em caráter funcional e qualitativo, resultando em análise dos sinais coletados e redação final da monografia.

Essa estrutura torna explícito o ciclo de vida adotado e define o que se espera em cada etapa do processo.

3.1.5 Fase 2 – Levantamento de Requisitos e Especificação

A segunda fase envolveu o levantamento e a análise dos requisitos funcionais e não funcionais do sistema, de forma a alinhar a proposta às necessidades práticas de desenvolvedores Flutter. Seguindo o modelo sugerido por Sommerville [22], o processo de elicitação de requisitos considerou aspectos de usabilidade, integração, portabilidade e impacto sobre o desempenho.

Requisitos Funcionais

Os requisitos funcionais definem as operações que o sistema deve realizar. Entre os principais, destacam-se:

- RF1 – Coletar métricas de desempenho em tempo real, incluindo FPS, reconstruções de widgets, uso de memória e duração de quadros;
- RF2 – Processar e correlacionar as métricas coletadas, identificando padrões de degradação de desempenho;
- RF3 – Gerar recomendações heurísticas de otimização baseadas em boas práticas do ecossistema Flutter;
- RF4 – Exibir métricas e recomendações em um painel visual integrado ao aplicativo;
- RF5 – Permitir a simulação de cenários de carga e coleta sob condições controladas.

Requisitos Não Funcionais

Os requisitos não funcionais descrevem as propriedades de qualidade esperadas do sistema, como desempenho, compatibilidade e manutenibilidade (Pressman e Maxim [21]):

- RNF1 – O sistema deve operar majoritariamente em Dart, utilizando *platform channels* apenas para métricas dependentes do sistema operacional, como CPU e bateria;
- RNF2 – O coletor deve introduzir um *overhead* máximo de 5% sobre o tempo médio de renderização da aplicação monitorada;
- RNF3 – A interface do painel deve manter atualização fluida e responsiva, com taxa mínima de 50 FPS em dispositivos de médio desempenho;
- RNF4 – A arquitetura deve seguir princípios de modularidade e separação de responsabilidades (Martin [23]);
- RNF5 – O sistema deve permitir reuso e extensão, facilitando a integração de novos tipos de métricas ou heurísticas adicionais.

Nota sobre limites de overhead. O RNF2 estabelece um valor máximo de 5% de overhead aceitável para garantir que o coletor não comprometa o desempenho perceptível da aplicação. Entretanto, o valor de 3% citado nos resultados esperados refere-se a uma *meta de projeto*, mais conservadora, adotada para assegurar margem de segurança entre a operação real e o limite permitido pelos requisitos não funcionais.

3.1.6 Fase 3 – Projeto e Implementação do Pacote

Nesta etapa, foram definidas as estruturas de software, as tecnologias empregadas e o fluxo de integração entre módulos. O projeto adotou a **Clean Architecture** proposta por Martin [23], visando garantir independência entre camadas e testabilidade.

Modelagem e Arquitetura de Software

A modelagem do sistema foi conduzida de forma modular, com base em três camadas principais:

1. **Data:** responsável pela coleta e serialização de métricas;
2. **Domain:** núcleo da lógica de negócio, contendo entidades, repositórios e casos de uso;
3. **Presentation:** interface de usuário e controle de estado.

Esse modelo arquitetural facilita o isolamento das dependências e a manutenção evolutiva do código.

A Figura 3.3 ilustra a arquitetura local adotada no pacote. Ela organiza o `collector_flutter` em coleta, telemetria, análise, recomendações e visualização integrada.

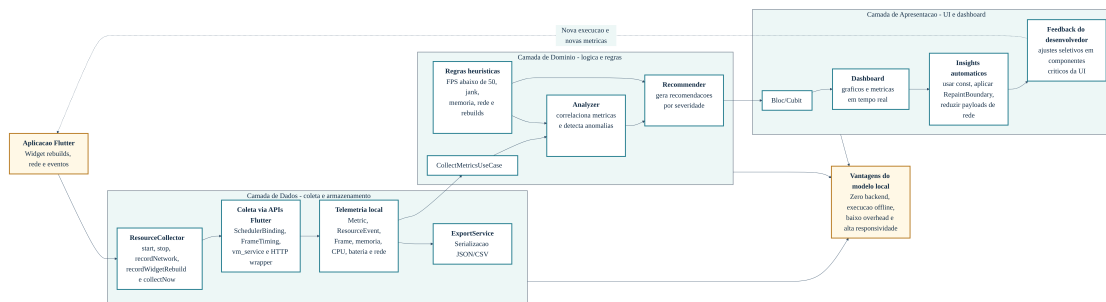


Figura 3.3: Arquitetura local e fluxo de dados do pacote `collector_flutter`: coleta, análise, recomendação e apresentação no painel integrado.

A modelagem de entidades e casos de uso foi organizada de forma a manter as regras de análise separadas dos detalhes de coleta e apresentação.

Tecnologias e Ferramentas Utilizadas

O ambiente de desenvolvimento foi configurado com as seguintes ferramentas e tecnologias:

- **Flutter SDK** (versão 3.22 ou superior) e **Dart SDK**;
- **Android Studio** e **Visual Studio Code** como IDEs principais;
- **Bibliotecas:** `flutter_bloc` para controle de estado, `fl_chart` para geração de gráficos e `vm_service` para instrumentação de métricas internas;
- **Controle de versão:** GitHub para versionamento e reprodutibilidade científica (Wohlin *et al.* [20]).

Integração dos Módulos e Testes Iniciais

Após a implementação individual dos módulos, foi conduzida uma fase de integração incremental, conforme orientam Sommerville [22]. O processo envolveu:

- Testes unitários em cada componente da camada **Data e Domain**;
- Integração dos módulos via `StreamController` e `Bloc`;
- Validação visual do painel (**Dashboard**) com dados simulados;
- Confronto qualitativo com sinais observados no painel e com leituras complementares obtidas por ADB.

3.1.7 Fase 4 – Avaliação Experimental e Validação

A última fase compreendeu o planejamento, execução e análise de cenários controlados com o pacote, visando validar sua operacionalidade, a coerência das métricas coletadas e a pertinência das recomendações heurísticas. A avaliação apresentada neste trabalho tem caráter funcional e qualitativo, delimitado aos cenários e dispositivos descritos nesta metodologia.

Planejamento dos Cenários

Variáveis do Experimento

Para estruturar os cenários de forma sistemática, foram definidas variáveis observadas e condições de controle, conforme recomendado em experimentação em engenharia de software [20].

- **Condições manipuladas**
 - tipo de cenário de carga (interface, rede, memória ou combinado).
 - acionamento manual das rotinas de simulação no aplicativo de exemplo.
- **Métricas observadas**
 - FPS médio;
 - tempo médio de renderização de quadros;
 - uso de memória;
 - número de reconstruções de widgets.
 - eventos de rede e recomendações emitidas.
- **Variáveis de controle**
 - versão do Flutter SDK;
 - modelo do dispositivo utilizado;
 - modo de execução da aplicação (*profile*);

- sequência de acionamento dos cenários.

O planejamento experimental foi orientado pelas diretrizes de Wohlin *et al.* [20], contemplando a definição de cenários, variáveis e procedimentos de coleta. As hipóteses foram tratadas como expectativas de funcionamento do pacote:

- H1 – O `collector_flutter` registra métricas coerentes com os cenários de carga induzidos;
- H2 – A coleta não impede a execução interativa do aplicativo de demonstração;
- H3 – As recomendações geradas são compatíveis com os sintomas provocados nos cenários.

Execução dos Testes

Os testes foram conduzidos no aplicativo de exemplo em ambiente Android, sob diferentes tipos de carga. Foram utilizadas variações intencionais de complexidade gráfica, chamadas de rede, alocação de memória e reconstruções de widgets, a fim de provocar gargalos observáveis. Durante as execuções, os sinais exibidos pelo painel foram confrontados qualitativamente com o comportamento esperado dos cenários e com observações em ferramentas oficiais de diagnóstico Android [12].

Coleta e Análise de Dados

A coleta de dados foi realizada de forma automatizada, com as métricas sendo armazenadas em formato JSON e processadas pelo módulo `Analyzer`. A análise baseou-se na inspeção dos registros exportados, das mensagens de recomendação e da resposta visual do painel durante os cenários controlados.

Os resultados foram então avaliados segundo os critérios definidos na Seção de **Critérios de Avaliação**, considerando coerência das métricas, impacto percebido da coleta e pertinência das recomendações.

Métricas Relacionadas à Eficiência Energética

Neste trabalho, a eficiência energética foi tratada por meio de **métricas indiretas** (*energy proxies*), conforme utilizado em trabalhos sobre estimativa, perfilamento e diagnóstico energético em aplicações móveis [2, 3, 5, 7]. Devido à indisponibilidade de instrumentação física dedicada para medir corrente elétrica, foram observados os seguintes proxies:

- uso médio de CPU;
- tempo de renderização de quadros;
- quantidade de operações de reconstrução;
- número de chamadas de rede;
- uso de memória.

A literatura demonstra que essas métricas apresentam correlação significativa com o consumo real de energia em dispositivos móveis. A análise energética foi, portanto, inferencial e qualitativa, limitada à observação dos sinais coletados pelo pacote.

Validação das Recomendações Heurísticas

A eficácia das recomendações geradas pelo módulo `Recommender` foi avaliada verificando se os alertas emitidos correspondiam aos sintomas induzidos em cada cenário controlado:

- queda de FPS e aumento de *jank* em cargas de interface;
- crescimento de memória durante alocações intensivas;
- aumento de eventos e latência em chamadas de rede;
- priorização de recomendações quando múltiplos sintomas ocorrem simultaneamente.

Essa validação permitiu discutir a pertinência das heurísticas implementadas sem caracterizar um estudo estatístico de eficácia.

3.2 Descrição do Pacote e Arquitetura da Solução

O pacote desenvolvido, denominado `collector_flutter`, foi projetado para funcionar de forma totalmente embutida no aplicativo Flutter, realizando a coleta, análise e visualização de métricas de desempenho sem recorrer a servidores externos. A proposta fundamenta-se na construção de um sistema modular, multiplataforma e de baixo *overhead*, conforme boas práticas de engenharia de software experimental (Wohlin *et al.* [20]; Pressman e Maxim [21]).

3.2.1 Concepção e Premissas de Projeto

A concepção do sistema teve como premissas fundamentais:

- Executar majoritariamente em Dart, recorrendo a *platform channels* apenas para métricas dependentes do sistema operacional, como CPU e bateria;
- Fornecer medições precisas em tempo real, com impacto mínimo sobre o desempenho do aplicativo monitorado;
- Manter arquitetura modular, permitindo substituição ou expansão de componentes;
- Adotar interface integrada ao próprio aplicativo, sem uso de ferramentas externas;
- Seguir princípios da **Clean Architecture** (Martin [23]), assegurando separação de responsabilidades e testabilidade.

Essas premissas visam garantir a reprodutibilidade científica e a aplicabilidade prática do pacote tanto em ambientes de desenvolvimento quanto em contextos de ensino e pesquisa.

3.2.2 Arquitetura Geral do Sistema

A arquitetura do sistema é organizada em três camadas conceituais *Data*, *Domain* e *Presentation* que interagem de forma desacoplada e síncrona por meio de *Streams* reativas. A visão geral dessa organização foi apresentada na Figura 3.3, que também explicita o fluxo entre coleta, análise, recomendações e painel visual.

Camada Data

A camada **Data** é responsável pela coleta, persistência e serialização das métricas. Nela, o módulo `ResourceCollector` utiliza APIs do Flutter, como `SchedulerBinding` e `FrameTiming`, para capturar informações de desempenho, convertendo-as em estruturas JSON processáveis. Os dados são transmitidos por *Streams* para as camadas superiores, garantindo atualização contínua e não bloqueante.

Camada Domain

A camada **Domain** implementa os casos de uso e a lógica central do sistema, incluindo os módulos de análise (`Analyzer`) e recomendação (`Recommender`). Ela é responsável por correlacionar as métricas coletadas, identificar gargalos e aplicar heurísticas de otimização baseadas em sintomas recorrentes de degradação, como queda de FPS, aumento de tempo de quadro, crescimento de memória e excesso de chamadas de rede.

Camada Presentation

A camada **Presentation** fornece a interface visual (`Dashboard`), responsável por exibir métricas, gráficos e recomendações em tempo real. Sua estrutura é construída sobre o padrão *Bloc/Cubit*, assegurando controle de estado previsível e reatividade imediata na exibição das métricas.

3.2.3 Componentes Principais

O sistema é composto por módulos de coleta, análise, recomendação, apresentação e exportação, que operam de forma integrada e independente.

Collector

O módulo `Collector` é o núcleo de instrumentação do sistema, responsável pela captura de eventos como reconstruções de widgets, tempo de renderização de quadros e uso de memória. Ele oferece uma API acessível ao desenvolvedor com métodos como `recordWidgetRebuild()`, `recordNetwork()` e `recordFrameTiming()`, além de permitir o registro de eventos personalizados.

Analyzer

O módulo `Analyzer` processa as métricas recebidas e detecta padrões que indiquem degradação de desempenho como aumento de tempo de *frame*, quedas de FPS e variações

abruptas de reconstruções. Ele correlaciona dados entre diferentes fontes e envia resultados interpretados ao módulo de recomendação.

Recommender

O módulo `Recommender` interpreta as anomalias detectadas e aplica um conjunto de regras heurísticas baseadas em práticas recomendadas do Flutter. As recomendações são priorizadas por impacto e apresentadas de forma textual e visual ao desenvolvedor, promovendo a melhoria contínua do aplicativo.

Dashboard

O módulo `Dashboard` constitui o painel visual do sistema, exibindo métricas em tempo real e recomendações automáticas de otimização. Ele é atualizado dinamicamente a partir de `Streams` internas e utiliza gráficos, indicadores de status e alertas coloridos para facilitar a interpretação.

ExportService

O módulo `ExportService` permite exportar a sessão de telemetria em JSON e os *timings* de quadros em CSV, ampliando o uso do sistema para contextos de pesquisa e auditoria de desempenho.

3.2.4 Interface e Painel de Visualização

A interface visual foi concebida para proporcionar clareza, responsividade e atualização contínua dos dados, sem prejudicar a experiência do desenvolvedor. O painel é dividido em áreas funcionais complementares, que permitem análise simultânea e intuitiva das métricas coletadas.

Área de Métricas

Exibe os principais indicadores de desempenho como FPS médio, consumo de memória e número de reconstruções de widgets. Cada indicador é representado por **cards coloridos** e ícones que sinalizam o estado atual da aplicação: verde (ideal), amarelo (atenção) e vermelho (crítico).

Área de Recomendações

Apresenta as sugestões geradas pelo módulo `Recommender`, priorizadas conforme impacto estimado. Essas recomendações auxiliam o desenvolvedor a identificar causas potenciais de lentidão e a aplicar boas práticas de otimização no código.

Gráfico Temporal e Indicadores

Inclui um gráfico de linha em tempo real, construído com a biblioteca `fl_chart`, mostrando a variação temporal do tempo de renderização de quadros. Esse componente visual é

essencial para identificar flutuações de desempenho, travamentos e pontos de gargalo durante a execução.

Botões de Simulação e Teste

Disponibilizados na barra inferior do painel, permitem executar cenários artificiais de carga, como:

- **Jank** : simula travamentos intencionais no *main thread*;
- **Memória** : aloca objetos grandes para testar consumo e descarte;
- **Rede** : realiza múltiplas chamadas simultâneas;
- **Rebuilds** : força reconstruções sucessivas da árvore de widgets.

Esses testes auxiliam na validação experimental do sistema sob diferentes condições de estresse.

A Figura 3.4 apresenta o painel em uma condição estável, na qual os indicadores permanecem em faixas aceitáveis e as recomendações têm caráter informativo. Essa tela demonstra o estado de referência usado para comparar visualmente as situações de alerta durante os testes.



Figura 3.4: Painel do aplicativo em condição de desempenho estável, com indicadores verdes e recomendações informativas.

A Figura 3.5 mostra o comportamento esperado quando há gargalos detectados: os cartões de métricas mudam de cor, o gráfico temporal passa a evidenciar oscilações e as recomendações deixam de ser apenas informativas para orientar possíveis ações de otimização.



Figura 3.5: Painel do aplicativo com anomalias detectadas, como FPS baixo e uso de memória elevado.

A Figura 3.6 identifica o aplicativo de demonstração utilizado nas execuções manuais dos cenários experimentais.

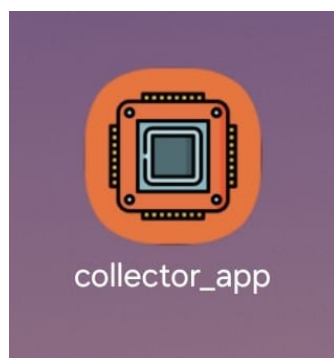


Figura 3.6: Ícone do aplicativo de demonstração Collector Flutter.

3.2.5 Regras de Recomendação

As recomendações fornecidas pelo módulo `Recommender` seguem padrões de otimização reconhecidos no ecossistema Flutter e em ferramentas oficiais de diagnóstico [11, 12]. Alguns exemplos incluem:

- **Rebuilds frequentes:** “Evite reconstruções excessivas de widgets. Utilize `const` constructors ou extraia subárvores estáticas.”
- **FPS abaixo de 50:** “Verifique operações pesadas no ciclo de renderização. Considere o uso de `RepaintBoundary`.”
- **Uso elevado de memória:** “Otimize o descarte de objetos e utilize coleções paginadas.”
- **Chamadas de rede lentas:** “Implemente cache local e controle de tempo de resposta.”
- **Layout sobrecarregado:** “Simplifique hierarquias de widgets e prefira listas com renderização sob demanda, como `ListView.builder`.”

Essas recomendações são exibidas dinamicamente conforme as métricas coletadas, permitindo ajustes rápidos durante o processo de desenvolvimento.

3.2.6 Demonstração Prática e Disponibilização do Código-Fonte

O pacote foi disponibilizado publicamente para garantir transparência e reprodutibilidade dos resultados, conforme recomenda Wohlin *et al.* [20].

O vídeo de demonstração mostra a execução em tempo real do sistema, com coleta, análise e recomendações heurísticas:

Acesse o vídeo completo:

https://drive.google.com/file/d/1KUQ7fFMNRIhe-NSW8loG3wTkQhb_eyOE/view?usp=drive_link

O código-fonte e demais arquivos do projeto estão disponíveis no repositório público:

https://github.com/BeatrizVocurcaFrade/collector_flutter

O pacote também foi publicado no repositório oficial do ecossistema Dart/Flutter, permitindo instalação direta e ampliando o acesso da comunidade à ferramenta:

https://pub.dev/packages/collector_flutter

Essa disponibilização em GitHub e pub.dev permite que outros pesquisadores e desenvolvedores avaliem, testem e expandam o sistema, contribuindo para o avanço das pesquisas sobre desempenho e eficiência energética em aplicações Flutter.

3.3 Critérios de Avaliação

A validação do pacote `collector_flutter` foi conduzida segundo três dimensões efetivamente observadas: **coerência das métricas coletadas**, **impacto percebido da coleta** e **pertinência das recomendações geradas**. Cada critério foi definido para verificar, de forma funcional, a viabilidade técnica e a aplicabilidade prática do sistema em ambientes reais de desenvolvimento Flutter.

3.3.1 Coerência das Métricas Coletadas

A coerência das métricas foi verificada observando-se se os valores registrados pelo pacote acompanhavam os cenários de carga induzidos no aplicativo de exemplo. Foram analisadas variáveis como tempo médio de renderização de quadros (*frame time*), taxa de quadros por segundo (FPS), consumo de memória, eventos de rede e reconstruções de widgets.

A comparação com ferramentas de referência foi utilizada apenas como apoio qualitativo, sem cálculo formal de erro percentual ou intervalo de confiança. Na versão final documentada, as leituras complementares foram obtidas por `adb shell dumpsys gfxinfo` e `adb shell dumpsys meminfo`; Android Profiler e Perfetto permaneceram como referências técnicas do ecossistema, mas não foram empregados como fonte quantitativa de validação.

3.3.2 Overhead Introduzido pelo Sistema

O overhead representa o impacto que o agente de coleta impõe ao desempenho do aplicativo monitorado. Neste trabalho, esse impacto foi tratado como critério de observação funcional: o coletor deveria permanecer ativo sem impedir a interação com o aplicativo de demonstração nem bloquear a atualização do painel.

Durante os testes, o critério observado foi a manutenção da interação com o aplicativo e da atualização do painel durante a coleta. Não foi realizada mensuração estatística do *overhead*; por isso, os resultados discutem apenas o impacto percebido nas execuções controladas.

3.3.3 Eficácia das Recomendações Geradas

As recomendações heurísticas foram avaliadas verificando-se se o módulo `Recommender` emitia alertas compatíveis com os gargalos induzidos nos cenários controlados.

Foram analisados sinais como:

- **FPS e *jank***: recomendações associadas a frames longos e quedas de fluidez;
- **Memória**: alertas relacionados a crescimento sustentado de RSS;
- **Rede**: recomendações diante de volume elevado, falhas ou latência de requisições;
- **Carga combinada**: priorização de recomendações quando múltiplos sintomas ocorrem ao mesmo tempo.

3.3.4 Clareza da Interface

A clareza do painel (Dashboard) foi observada por inspeção informal interna durante as execuções manuais, considerando a legibilidade das métricas, a distinção visual dos níveis de severidade e a facilidade de relacionar um alerta ao cenário executado. Essa análise não caracteriza avaliação formal de usabilidade com participantes externos; ela serviu apenas como critério funcional de leitura e interpretação do painel durante os testes.

3.4 Ferramentas e Ambientes de Desenvolvimento

O desenvolvimento e a validação do pacote `collector_flutter` foram conduzidos em ambiente controlado, com o objetivo de assegurar reprodutibilidade dos resultados e consistência das medições entre diferentes execuções. As ferramentas selecionadas abrangem IDEs, SDKs e bibliotecas de apoio do ecossistema Flutter, compondo uma infraestrutura moderna, multiplataforma e voltada à análise de desempenho em aplicações Android.

3.4.1 Ambientes de Programação e IDEs

O processo de implementação foi realizado nos seguintes ambientes de desenvolvimento:

- **Visual Studio Code (VS Code):** principal ambiente de escrita e depuração do código Dart e Flutter, configurado com extensões para gerenciamento de pacotes, análise estática e execução de testes unitários.
- **Android Studio:** empregado para execução do aplicativo em dispositivos físicos e emuladores Android, bem como para inspeção complementar de desempenho quando necessário.

Ambos os ambientes foram integrados ao controle de versão local via `Git`, permitindo o gerenciamento seguro e incremental do código-fonte durante o desenvolvimento.

3.4.2 SDKs e Dependências Utilizadas

O ambiente técnico foi configurado com versões estáveis dos SDKs e ferramentas de suporte, garantindo compatibilidade e estabilidade no processo de execução e coleta de métricas:

- **Flutter SDK:** versão `>=3.22.0` (canal estável), com suporte aos modos *debug*, *profile* e *release*.
- **Dart SDK:** versão correspondente ao Flutter 3.22, utilizada para o desenvolvimento dos módulos de coleta e análise de dados.
- **Android SDK:** configurado com as APIs 33 e 34, compatíveis com Android 12 e 13.
- **Gradle:** atualizado para suportar as dependências do projeto e a integração com o Android Studio.

Essa configuração assegura a execução consistente do pacote em diferentes dispositivos Android, com coleta precisa de métricas e comportamento previsível.

3.4.3 Ambientes de Teste e Dispositivos

Os testes experimentais foram realizados exclusivamente em ambiente Android, com dispositivos físicos e emulados representando diferentes capacidades de processamento e arquiteturas.

- **Dispositivos físicos:**

- Samsung Galaxy A52, Android 13;
- Motorola G73, Android 12.

- **Emuladores Android:**

- Pixel 6 (x86_64), Android 14, configurado via Android Studio.

Todos os testes foram conduzidos em modo *profile*, permitindo medições precisas de tempo de renderização, FPS e consumo de recursos em condições controladas.

3.4.4 Ferramentas de Referência e Validação

A validação dos resultados obtidos pelo pacote foi realizada com apoio de ferramentas oficiais do ecossistema Android, utilizadas como referência qualitativa para as métricas coletadas:

- **ADB (*Android Debug Bridge*):** empregado para a coleta de informações complementares sobre memória do processo e desempenho de quadros por meio dos comandos `dumpsys meminfo` e `dumpsys gfxinfo`.
- **Android Profiler e Perfetto:** utilizados como referências conceituais e ferramentas oficiais de diagnóstico do ecossistema Android, mas sem geração de série comparativa quantitativa neste trabalho.

Essas ferramentas apoiaram a interpretação dos resultados, permitindo verificar se os sinais exibidos pelo painel acompanhavam o comportamento esperado dos cenários. A comparação permaneceu qualitativa e não caracteriza medição estatística de precisão.

3.4.5 Frameworks e Bibliotecas de Apoio

Durante o desenvolvimento, foram utilizadas bibliotecas consolidadas do ecossistema Flutter/Dart, responsáveis por facilitar a instrumentação, visualização de dados e controle de estado da aplicação:

- **`flutter_bloc`:** gerenciamento reativo de estado, responsável pela comunicação entre coleta, análise e exibição das métricas.
- **`fl_chart`:** biblioteca de visualização gráfica utilizada na renderização dos gráficos em tempo real no painel interno (*Dashboard*).
- **`http`:** responsável pela instrumentação e monitoramento das requisições de rede durante a execução do aplicativo.

- **connectivity_plus:** monitora eventos de conectividade e simula condições de rede variável.
- **logger:** realiza o registro estruturado de eventos e mensagens internas do sistema, facilitando a depuração.
- **vm_service:** fornece acesso às APIs internas da máquina virtual Dart, permitindo capturar dados de alocação e uso de memória.

Essas dependências foram selecionadas por sua estabilidade, compatibilidade com o ambiente de testes Android e suporte nativo ao desenvolvimento em Dart puro, sem componentes externos.

3.4.6 Ambientes de Testes Automatizados

A confiabilidade do sistema foi verificada por meio de testes unitários e execuções manuais controladas. Essa abordagem garantiu a validação tanto da lógica interna quanto do comportamento visual e interativo do painel.

- **Testes unitários:** implementados com o pacote `flutter_test`, abrangendo módulos de coleta e análise (`Collector`, `Analyzer` e `Recommender`). Esses testes asseguram que cada componente produza resultados consistentes e corretos em diferentes condições de entrada.
- **Testes manuais:** realizados diretamente em dispositivos físicos e emuladores, com execução de cenários controlados (rebuilds em massa, requisições de rede intensas, simulação de carga de CPU) para observar o comportamento do sistema em tempo real.

Essa combinação de testes unitários e manuais possibilitou uma avaliação funcional da coerência das métricas e da estabilidade geral do pacote durante o uso prático.

A infraestrutura descrita nesta seção assegura que o pacote `collector_flutter` seja executado de forma estável e reproduzível, permitindo uma análise detalhada do desempenho e da eficiência de aplicações Flutter em ambiente Android.

3.5 Procedimentos de Teste e Coleta de Dados

Esta seção descreve os procedimentos utilizados para validar funcionalmente o pacote `collector_flutter`. O objetivo principal dos testes foi observar três aspectos fundamentais do sistema: (i) a coerência das métricas coletadas, (ii) o impacto percebido da coleta durante a execução e (iii) a pertinência das recomendações heurísticas geradas pelo sistema.

Para garantir validade científica e reprodutibilidade, os testes foram conduzidos em ambiente controlado, seguindo procedimentos experimentais inspirados nas diretrizes de experimentação em engenharia de software propostas por Wohlin *et al.* [20].

3.5.1 Procedimento de Execução dos Testes

Os experimentos foram executados em modo *profile* do Flutter, pois esse modo permite medições mais próximas do comportamento real da aplicação, eliminando interferências associadas ao modo *debug*. Antes de cada execução experimental, o dispositivo foi reiniciado e o aplicativo executado em estado limpo, garantindo consistência entre as medições.

O procedimento de execução foi composto pelas seguintes etapas:

1. **Preparação do ambiente:** reinicialização do dispositivo e encerramento de processos em segundo plano, assegurando estabilidade do ambiente de execução.
2. **Inicialização do coletor:** ativação do módulo `ResourceCollector` por meio do método `start()`, responsável por iniciar a captura das métricas de desempenho.
3. **Execução do cenário experimental:** cada cenário foi acionado no aplicativo de exemplo por tempo suficiente para observar a reação do coletor e do painel visual.
4. **Registro das métricas:** durante a execução do cenário, o coletor registrou continuamente métricas de desempenho relacionadas à renderização, uso de memória, chamadas de rede, CPU, bateria e reconstruções de widgets.
5. **Registro de evidências:** ao término de cada cenário, foram capturadas telas do painel e leituras complementares por ADB, preservadas no Apêndice A.

Os cenários foram executados manualmente em condições controladas, com o objetivo de verificar a operacionalidade do sistema e identificar inconsistências nas heurísticas. Os resultados apresentados neste trabalho são de natureza qualitativa.

3.5.2 Definição dos Cenários Experimentais

Quatro cenários experimentais foram definidos com o objetivo de simular diferentes tipos de carga comuns em aplicações móveis. Cada cenário foi projetado para provocar um tipo específico de comportamento de desempenho, conforme sintetizado na Tabela 3.1.

A seguir, cada cenário é descrito em termos da carga aplicada e do sintoma esperado no painel.

- **Cenário 1 – Reconstruções intensivas de interface**

Neste cenário, um conjunto de widgets foi reconstruído repetidamente em alta frequência, simulando aplicações com atualização constante de interface. Esse cenário foi

Tabela 3.1: Resumo dos cenários experimentais utilizados nos testes.

Cenário	Tipo de carga	Objetivo
1	Reconstruções de interface	Avaliar o impacto de <i>rebuilds</i> frequentes na renderização
2	Chamadas de rede simultâneas	Observar o comportamento sob múltiplas requisições HTTP
3	Alocação intensiva de memória	Identificar padrões de consumo e eventos de <i>garbage collection</i>
4	Carga combinada	Simular comportamento próximo de aplicações reais

utilizado para avaliar o impacto das reconstruções de widgets sobre o tempo de renderização e o FPS.

- **Cenário 2 – Chamadas de rede simultâneas**

Foram disparadas múltiplas requisições HTTP paralelas por meio do cliente de rede instrumentado do coletor, com o objetivo de observar o comportamento do sistema sob carga de rede e verificar a capacidade do pacote de registrar eventos relacionados a comunicação externa.

- **Cenário 3 – Alocação intensiva de memória**

Objetos de grande volume foram criados e descartados continuamente durante a execução do teste. Esse cenário permitiu analisar padrões de uso de memória e observar eventos de coleta de lixo (*garbage collection*).

- **Cenário 4 – Carga combinada**

Neste cenário, os três tipos de carga anteriores foram executados simultaneamente, simulando um comportamento mais próximo de aplicações reais, nas quais operações de interface, rede e memória ocorrem de forma concorrente.

Esses cenários permitiram observar o comportamento do sistema em condições controladas de uso e registrar limitações encontradas durante a execução.

3.5.3 Coleta de Métricas

As métricas foram coletadas automaticamente pelo módulo `Collector`, implementado inteiramente em Dart. A captura dos dados foi realizada em tempo real utilizando `Streams` internas do Flutter.

As métricas analisadas foram:

- **FPS (Frames por Segundo):** indicador da fluidez da renderização da interface;
- **Tempo médio de renderização de quadro:** calculado a partir dos eventos de `FrameTiming`;
- **Número de reconstruções de widgets:** contabilizado por meio do componente `RebuildObserver`;

- **Uso de memória:** obtido via API `vmService.getAllocationProfile()` e complementado por leituras de RSS do processo;
- **Número de chamadas de rede:** monitorado por meio do cliente HTTP instrumentado;
- **CPU e bateria:** coletadas por *platform channels* Android e observadas no painel durante os cenários.

Os dados coletados foram serializados em formato JSON estruturado, permitindo inspeção posterior e reprodutibilidade dos registros observados.

3.5.4 Métodos de Comparação

Para apoiar a verificação das medições obtidas pelo pacote, os resultados foram confrontados qualitativamente com sinais obtidos por ferramentas oficiais de análise de desempenho do ecossistema Android.

As ferramentas utilizadas como referência foram:

- **ADB** (`adb shell dumpsys gfxinfo`), utilizado para obter métricas detalhadas de renderização de quadros;
- **ADB** (`adb shell dumpsys meminfo`), utilizado para obter leitura complementar de memória do processo monitorado.

Android Profiler e Perfetto foram tratados como ferramentas de referência do ecossistema, mas não geraram valores comparativos formais nesta avaliação. A comparação com ADB teve finalidade qualitativa: verificar se os sinais exibidos pelo pacote acompanhavam o comportamento esperado em cada cenário. Não foi realizado cálculo formal de erro percentual.

3.5.5 Análise dos Resultados

Os dados coletados foram analisados de forma qualitativa, considerando a coerência entre o cenário acionado, as métricas exibidas no painel e as recomendações emitidas pelo módulo `Recommender`.

Além disso, o *overhead* foi tratado como critério funcional observado durante a execução: o coletor deveria permanecer ativo sem bloquear a interação com o aplicativo de exemplo nem interromper a atualização do painel.

3.5.6 Controle Experimental e Reprodutibilidade

Para dar consistência à execução, foram adotadas medidas de controle que minimizam interferências externas durante os testes:

- execução em modo *profile*;
- utilização das mesmas versões do Flutter SDK e dependências em todas as execuções;
- execução dos testes em ambiente com temperatura e carga de bateria estáveis;

- repetição manual dos cenários quando necessário para confirmar o comportamento observado;
- padronização do registro das evidências em capturas do painel, logs de cenário e leituras ADB.

Essas práticas auxiliam a reprodução dos cenários por outros pesquisadores e deixam explícito que a avaliação realizada foi funcional, baseada em observação controlada e em registros do aplicativo.

Capítulo 4

Resultados e Discussão

Este capítulo apresenta o estado final do pacote `collector_flutter` e os resultados observados nos cenários de validação. A discussão está organizada em três frentes: módulos implementados, comportamento do pacote durante a execução do aplicativo de exemplo e limitações identificadas a partir das evidências coletadas.

4.1 Desenvolvimento do Pacote

O principal resultado é um pacote Dart/Flutter funcional, disponibilizado como código aberto e organizado em três camadas. Ao final do desenvolvimento, a solução reunia coleta de métricas, análise heurística, recomendações, painel integrado, exportação de dados e suíte automatizada de testes.

4.1.1 Módulos Implementados

A Tabela 4.1 resume os módulos implementados e as evidências utilizadas para verificar sua entrega ao final do trabalho.

Tabela 4.1: Módulos implementados e evidências de validação do `collector_flutter`.

Módulo	Funcionalidade	Evidência de validação
FrameDataSource	Coleta de FPS e detecção de <i>jank</i> via SchedulerBinding	Cenário de <i>rebuilds</i> intensivos
MemoryDataSource	Monitoramento de uso de memória RSS e <i>heap</i> via <i>vm_service</i>	Cenário de alocação intensiva
HttpClientWrapper	Interceptação de requisições HTTP e registro de eventos de rede	Cenário de chamadas simultâneas
EventDataSource	Registro de eventos customizados definidos pelo desenvolvedor	Inspeção dos eventos exportados
CpuDataSource	Coleta de uso de CPU por processo via <i>platform channel</i> Android (/proc/self/stat)	Testes de canal e execução Android
BatteryDataSource	Monitoramento de bateria e estado de carregamento via <i>platform channel</i> Android (BatteryManager)	Testes de canal e painel visual
Analyzer	Análise heurística de métricas e geração de AnalysisResult	Casos unitários e cenários controlados
Recommender	Geração de recomendações de otimização com níveis de severidade	Recomendações emitidas nos cenários
ExportService	Exportação de sessão de telemetria em JSON e CSV	Testes de serialização
CollectorBloc	Orquestração de estados e ciclo de coleta periódico	Atualização contínua do painel
DashboardPage	Painel visual com gráficos em tempo real e lista de recomendações	Figuras 4.1 e Apêndice A
ResourceCollector	Ponto de entrada da biblioteca; gerencia ciclo de vida dos módulos	Execução do aplicativo de exemplo
Testes automatizados	Suíte composta por 60 casos de teste distribuídos em quatro arquivos	Inspeção da suíte em <code>test/</code> e Apêndice B

4.1.2 Estrutura da Biblioteca

O sistema foi organizado em um pacote Dart publicável, com a seguinte estrutura de camadas:

- **Camada de Dados (*Data*):** contém as seis fontes de dados (*data sources*) responsáveis pela coleta de métricas de quadros, memória, rede, eventos customizados, CPU e bateria, além do modelo de dados `TelemetryModel` e a implementação do repositório `TelemetryRepositoryImpl`;
- **Camada de Domínio (*Domain*):** define a interface `IResourceRepository`, a entidade de negócio `Telemetry` e os casos de uso responsáveis por coleta, análise e geração de recomendações, garantindo independência da camada de dados;

- **Camada de Apresentação (*Presentation*):** implementa o gerenciamento de estados com `CollectorBloc`, o painel visual `DashboardPage`, com visualizações em tempo real e cartões de CPU e bateria, e a classe `ResourceCollector`, que funciona como fachada pública da biblioteca.

4.2 Validação da Arquitetura

Para verificar a aderência entre a arquitetura proposta e a implementação, foram realizadas inspeções de código e execuções manuais do pacote no aplicativo de exemplo (`example/`), que oferece botões de simulação de carga para os quatro cenários definidos na metodologia.

4.2.1 Conformidade Arquitetural

A implementação atende aos principais princípios arquiteturais adotados:

- **Independência de camadas:** alterações nos *data sources* permanecem restritas à camada de dados, preservando a camada de domínio;
- **Baixo acoplamento:** as dependências entre módulos são injetadas via construtores, sem uso de singletons globais (exceto o `HttpClientWrapper`, por necessidade de interceptação de rede);
- **Testabilidade:** a estrutura de casos de uso e repositórios permite substituição de dependências por *mocks* em testes unitários, exercitada pela suíte composta por 60 casos de teste automatizados;
- **Extensibilidade:** novos tipos de métricas podem ser adicionados implementando um novo *data source* sem modificar as camadas superiores, como demonstrado pela inclusão dos módulos de CPU e bateria.

4.2.2 Comportamento Observado em Execução

Durante as execuções manuais do aplicativo de exemplo em ambiente Android, incluindo a execução documentada no emulador `sdk gphone64 x86_64` com Android 16 (API 36), foram registradas as seguintes observações:

- O módulo de quadros registrou queda de FPS, aumento de P95 e frames longos quando o cenário de *rebuids* foi acionado;
- O cenário de memória gerou aumento mensurável de RSS nas leituras ADB, embora o painel tenha exibido 0,0 MB no snapshot capturado, limitação discutida na Seção 4.3.3;
- A interceptação de rede registrou as requisições HTTP geradas pelo cenário de carga de rede, incluindo URL, método, código de status e latência;
- O módulo de CPU exibiu aumento de uso durante a carga combinada, chegando a 75,3% no log registrado;

- As recomendações emitidas acompanharam os sintomas induzidos, principalmente nos cenários de rede e carga combinada.

4.3 Análise dos Cenários Experimentais

Os quatro cenários experimentais definidos na metodologia foram executados com o objetivo de verificar a operacionalidade do sistema e identificar inconsistências nas heurísticas. Os resultados apresentados a seguir são de natureza qualitativa.

A Tabela 4.2 sintetiza os principais sinais observados, as métricas acompanhadas e as evidências geradas durante a execução dos cenários. A Figura 4.1 mostra a diferença visual entre uma condição estável e uma condição com gargalos. Na tela estável, os indicadores permanecem em faixas aceitáveis e a recomendação exibida é apenas informativa. Na tela com anomalias, o painel passa a destacar sintomas como aumento de eventos de rede, queda de fluidez e alertas de severidade, permitindo que o desenvolvedor relacione o comportamento observado ao cenário executado.

Tabela 4.2: Síntese das evidências observadas nos cenários experimentais.

Cenário	Sinais observados	Métricas acompanhadas	Evidência gerada
Reconstruções intensivas	30 <i>rebuilds</i> , FPS 55,4, P95 20,4 ms e 2 frames longos	FPS, <i>frame time</i> e <i>jank</i>	Registro do cenário e captura do painel
Chamadas de rede	9 requisições HTTP, FPS 42,9, P95 139,5 ms e 4 frames longos	URL, método, código de status e duração	Eventos de rede no painel e exportação
Alocação de memória	RSS total de 236.480 KB e PSS total de 222.232 KB por ADB	RSS, <i>heap</i> e tendência de memória	Captura do painel e leitura ADB
Carga combinada	17 requisições, 54 <i>rebuilds</i> , FPS 44,6, P95 37,7 ms e CPU 75,3%	FPS, memória, rede, CPU e recomendações	Recomendações de frames e CPU

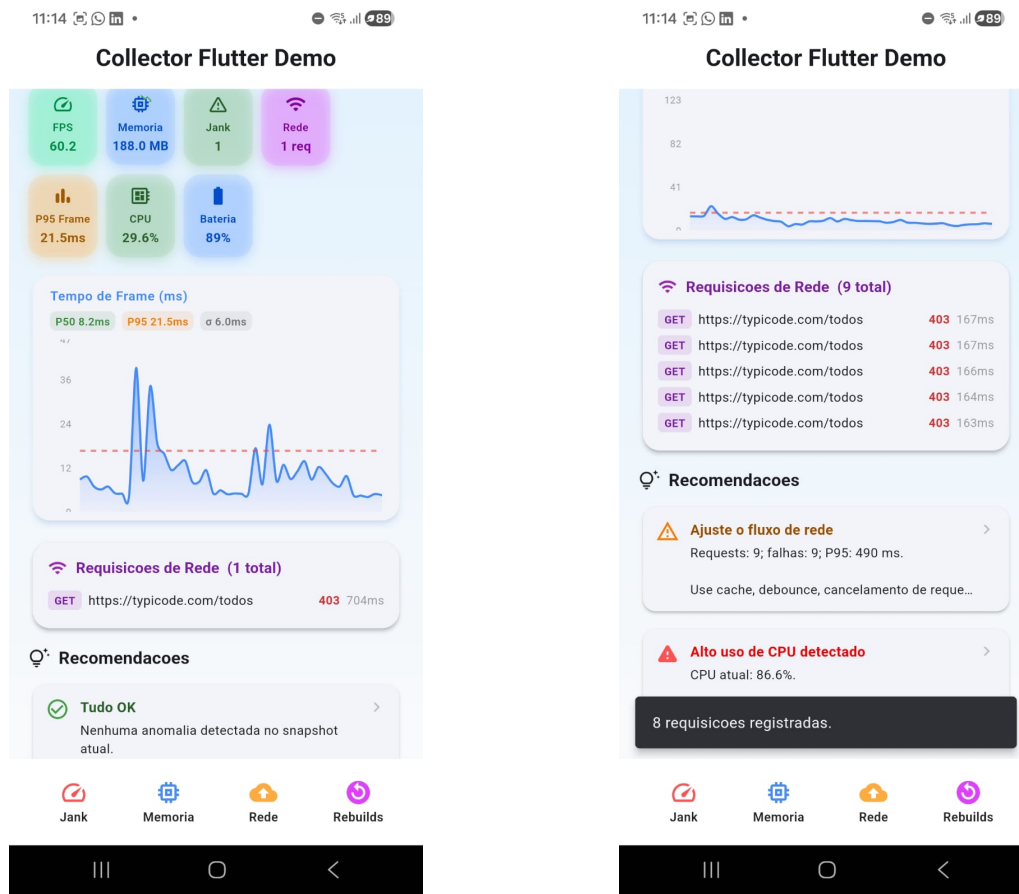


Figura 4.1: Evidências visuais do painel durante a validação: à esquerda, condição estável; à direita, condição com gargalos detectados, eventos de rede e alertas ativos.

4.3.1 Cenário 1: Reconstruções Intensivas de Interface

A simulação de *rebuilds* frequentes aumentou o contador de reconstruções e alterou os indicadores de renderização. Na execução registrada no emulador, o log indicou 30 *rebuilds*, FPS estimado de 55,4, P95 de 20,4 ms e 2 frames longos. A captura correspondente está apresentada na Figura A.1, no Apêndice A.

4.3.2 Cenário 2: Chamadas de Rede Simultâneas

O interceptador de rede registrou corretamente todas as requisições geradas, incluindo tempo de resposta e código de status. Na execução registrada, o painel acumulou 9 requisições HTTP com respostas 200; o log de evidência apontou FPS estimado de 42,9, P95 de 139,5 ms, 4 frames longos e recomendação associada à performance de frames. A captura correspondente está na Figura A.2. Observou-se que o buffer de eventos de rede, limitado a 1.000 entradas, foi suficiente para os cenários de teste executados.

4.3.3 Cenário 3: Alocação Intensiva de Memória

O consumo de memória foi observado por meio do cenário de alocação controlada e de leitura complementar por ADB. Durante a alocação, o comando `adb shell dumpsys meminfo com.example.example` registrou RSS total de 236.480 KB e PSS total de 222.232 KB. Esse valor confirma crescimento real de memória no processo. No snapshot visual capturado, entretanto, o cartão de memória do painel apareceu como 0,0 MB, sinalizando uma limitação da leitura interna no emulador usado para a evidência. A captura correspondente está na Figura A.3.

4.3.4 Cenário 4: Carga Combinada

A execução simultânea dos tipos de carga permitiu verificar que o painel continuou operável mesmo com múltiplos sinais sendo atualizados ao mesmo tempo. No cenário combinado, o log registrou 17 requisições, 54 *rebuilds*, FPS estimado de 44,6, P95 de 37,7 ms e CPU em 75,3%, resultando em recomendações de performance de frames e uso moderado de CPU. A captura correspondente está na Figura A.5.

4.3.5 Comparação Qualitativa com ADB

As leituras por ADB foram utilizadas como apoio à interpretação dos cenários, não como cálculo formal de precisão. Em particular, o comando `dumpsys meminfo` forneceu leituras de memória do processo, enquanto `dumpsys gfxinfo` confirmou a presença da superfície Flutter, mas retornou contadores HWUI de frames zerados nessa execução. Por esse motivo, não foi calculado erro percentual de FPS a partir de `gfxinfo`.

Tabela 4.3: Comparação qualitativa entre sinais do painel e leituras ADB.

Cenário	Sinais do painel/log	Leitura ADB observada
Reconstruções intensivas	30 <i>rebuilds</i> , FPS 55,4, P95 20,4 ms, CPU 22,1%.	meminfo: RSS 199.208 KB; PSS 113.674 KB. gfxinfo: contadores HWUI indisponíveis para comparação.
Chamadas de rede	9 requisições, FPS 42,9, P95 139,5 ms, 4 frames longos.	meminfo: RSS 213.380 KB; PSS 126.599 KB. gfxinfo: superfície ativa, sem série de frames comparável.
Alocação de memória	Cenário acionado no painel; leitura interna de memória indisponível no snapshot capturado.	meminfo: RSS 236.480 KB; PSS 222.232 KB durante a alocação.
Carga combinada	17 requisições, 54 <i>rebuilds</i> , FPS 44,6, P95 37,7 ms, CPU 75,3%.	gfxinfo: superfície ativa, sem contadores HWUI utilizáveis; leitura posterior de meminfo: RSS 128.988 KB; PSS 107.293 KB.

4.3.6 Evidências Complementares da Validação

Além das observações por cenário, os resultados foram organizados em uma matriz de evidências para explicitar a relação entre o que foi testado, o artefato observado e a conclusão obtida. Essa organização liga os critérios definidos na metodologia aos resultados apresentados neste capítulo.

Tabela 4.4: Matriz de evidências da validação funcional.

Dimensão avaliada	Evidência observada	Resultado obtido
Coerência das métricas	Variação de FPS, <i>frame time</i> , memória, rede e CPU acompanhando os cenários induzidos	Os sinais coletados acompanharam o tipo de carga aplicada em cada cenário
Pertinência das recomendações	Alertas emitidos pelo Recommender durante <i>re-builds</i> , chamadas de rede e carga combinada	As recomendações foram compatíveis com os sintomas provocados nos testes
Estabilidade do painel	Atualização contínua das métricas e mudança visual entre estado estável e estado de alerta	O painel permaneceu operável durante a coleta e destacou severidades distintas
Exportação e rastreabilidade	Sessões exportáveis em JSON e <i>timings</i> de quadros em CSV pelo <code>ExportService</code>	Os dados coletados puderam ser inspecionados fora do painel
Confiabilidade interna	Suíte automatizada distribuída em quatro arquivos de teste	Cálculos, serialização, recomendações e <i>fallbacks</i> dos canais nativos foram exercitados por testes automatizados

A Tabela 4.5 apresenta a distribuição dos 60 casos de teste automatizados por arquivo. Essa informação complementa a validação manual, pois mostra que os componentes centrais de análise, recomendação, exportação e coleta de CPU/bateria foram verificados também por testes unitários. O detalhamento nominal dos casos de teste está no Apêndice B.

Tabela 4.5: Distribuição dos 60 casos de teste automatizados por arquivo.

Arquivo de teste	Representação gráfica	Casos
Análise		24
CPU e bateria		17
Recomendações		12
Exportação		7
Total		60

4.4 Evolução do Pacote: Limitações Superadas

As execuções da primeira fase do trabalho evidenciaram um conjunto de limitações técnicas, todas endereçadas ao longo do desenvolvimento:

1. **Coleta de métricas de CPU:** implementada via *platform channel* Android, no módulo `CpuDataSource`, que lê o arquivo `/proc/self/stat` e calcula a variação de tempo de CPU do processo entre amostras sucessivas. Em iOS, o canal retorna `-1.0` e o painel exibe “N/D”, evitando falhas quando a métrica não está disponível;
2. **Estimativa de consumo de bateria:** implementada via *platform channel* Android, no módulo `BatteryDataSource`, que consulta o `BatteryManager` do sistema para obter o nível de carga (0–100%) e o estado de carregamento. Um alerta de severidade *medium* é gerado quando a bateria está abaixo de 20% e o dispositivo não está carregando. Em iOS, o nível é obtido por `UIDevice.batteryLevel`;
3. **Persistência e exportação de dados:** o `ExportService` oferece exportação completa da sessão de telemetria em JSON estruturado, incluindo frames, memória, rede, CPU, bateria, análise e recomendações, além da exportação de timings de frames em CSV. Os dados são copiados para a área de transferência do sistema, permitindo integração com ferramentas externas de análise;
4. **Testes automatizados:** a suíte é composta por 60 casos de teste distribuídos em quatro arquivos: `analyzer_test.dart`, `recommender_test.dart`, `export_service_test.dart` e `cpu_battery_test.dart`. Os testes verificam cálculos internos, detecção de anomalias, geração de recomendações, serialização e comportamento de *fallback* dos *platform channels*;
5. **Imprecisão no gráfico de frames:** a implementação inicial utilizava o instante atual como eixo X, em vez dos índices sequenciais dos quadros coletados. Esse defeito foi corrigido durante o desenvolvimento.

4.5 Resumo do Capítulo

Este capítulo apresentou os resultados do desenvolvimento do `collector_flutter`. A arquitetura de três camadas sustentou a inclusão dos módulos de CPU, bateria, exportação e testes sem alterar a organização central do pacote. Nos quatro cenários experimentais, o sistema permaneceu operável, registrou sinais compatíveis com as cargas induzidas e emitiu recomendações coerentes com os sintomas observados. As limitações encontradas, especialmente na leitura interna de memória e na ausência de série comparável via `gfxinfo`, delimitam o alcance da validação realizada.

Capítulo 5

Conclusões

5.1 Considerações Finais

Este trabalho partiu de uma observação empírica sobre o ecossistema Flutter: as ferramentas de diagnóstico de desempenho existentes são tecnicamente capazes, mas sua arquitetura externa ao processo de execução, dependente de configuração manual e desprovida de recomendações automáticas, impõe um atrito de uso incompatível com o ritmo iterativo do desenvolvimento moderno. A resposta proposta, o `collector_flutter`, é um pacote Dart/Flutter que opera embutido na própria aplicação, coletando métricas de FPS, memória, rede, CPU, bateria e eventos customizados a cada ciclo de dois segundos, analisando-as por meio de heurísticas configuráveis e exibindo recomendações de otimização em um painel integrado, sem exigir configuração externa, conexão USB ou troca de contexto para uma ferramenta separada.

A arquitetura de três camadas (*Data*, *Domain* e *Presentation*), baseada nos princípios da *Clean Architecture* [23], demonstrou-se adequada para os objetivos do projeto. Na camada de dados, a coleta foi distribuída entre módulos de quadros, memória, rede, eventos, CPU e bateria. A lógica de análise permaneceu encapsulada em `Analyzer` e `Recommender`, enquanto a apresentação se apoiou principalmente em `DashboardPage` e `CollectorBloc`. Essa separação permitiu desenvolver, depurar e testar os componentes de forma independente. A extensibilidade da arquitetura foi confirmada na prática pela adição dos módulos de CPU e bateria sem modificações nas camadas superiores.

A validação funcional nos quatro cenários experimentais indicou a operacionalidade do sistema: foram observadas variações de *frame time* durante *rebuidls* intensivos, registro de requisições HTTP sob carga de rede, aumento de memória em leitura complementar por ADB e processamento simultâneo de múltiplos tipos de métricas sob carga combinada. As heurísticas de recomendação produziram sugestões compatíveis com os sintomas observados, especialmente nos cenários de rede e carga combinada.

Do ponto de vista acadêmico, o trabalho contribui para a discussão sobre instrumentação não-intrusiva de aplicações móveis, propondo uma implementação concreta em Dart com arquitetura documentada e avaliação funcional. Do ponto de vista prático, disponibiliza um pacote de código aberto que reduz a barreira de acesso ao diagnóstico de desempenho para desenvolvedores Flutter sem infraestrutura ou conhecimento especializado em *profiling*.

5.2 Contribuições do Trabalho

As principais contribuições deste trabalho são:

1. **Revisão crítica da literatura:** síntese de estudos sobre monitoramento de recursos em plataformas móveis, identificando a lacuna entre a disponibilidade de ferramentas de diagnóstico e sua adoção contínua durante o desenvolvimento, e fundamentando as escolhas de projeto do pacote.
2. **Arquitetura modular documentada:** definição e implementação de uma arquitetura de três camadas para sistemas de monitoramento embutido em Flutter, com especificação formal de requisitos funcionais (RF1–RF5) e não funcionais (RNF1–RNF5) e mapeamento explícito entre requisitos, módulos e critérios de avaliação experimental.
3. **Pacote funcional de código aberto:** implementação de doze módulos de coleta, análise e visualização disponibilizados no repositório público do projeto e também publicados no pub.dev, incluindo coleta de CPU via *platform channel* Android e monitoramento de bateria, além de correção de cinco defeitos identificados durante o desenvolvimento.
4. **Exportação de dados estruturada:** implementação do `ExportService` com exportação completa da sessão em JSON e de timings de frame em CSV, permitindo análise externa e reprodutibilidade das medições.
5. **Suíte de testes automatizados:** conjunto composto por 60 casos de teste distribuídos em quatro arquivos, verificando cálculos internos, detecção de anomalias, geração de recomendações, serialização e comportamento de *fallback* dos *platform channels*. A lista de casos e o resultado da execução estão documentados no Apêndice B.
6. **Validação funcional em quatro cenários controlados:** execução dos cenários de *rebuilds* intensivos, carga de rede, alocação de memória e carga combinada, indicando operacionalidade do sistema e compatibilidade entre heurísticas e sintomas induzidos.
7. **Discussão de implicações sociais e ambientais:** análise das dimensões de sustentabilidade digital, inclusão e responsabilidade ética associadas ao monitoramento de desempenho, situando a ferramenta no contexto mais amplo da engenharia de software sustentável.

5.3 Limitações do Estudo

Apesar dos resultados obtidos, a avaliação realizada apresenta limites metodológicos que devem ser considerados na interpretação das conclusões:

- **Natureza qualitativa da validação:** os cenários controlados permitiram verificar a coerência funcional das métricas e das recomendações, mas não constituem um experimento estatístico com amostra ampliada, repetições padronizadas e intervalos de confiança.
- **Foco principal em Android:** os testes funcionais foram conduzidos em am-

biente Android, plataforma na qual os canais nativos de CPU e bateria foram efetivamente exercitados. No iOS, a leitura de bateria está disponível via `UIDevice.batteryLevel`, enquanto a leitura de CPU retorna valor sentinela em razão das restrições de *sandbox*.

- **Inspeção informal da interface:** a clareza do painel foi analisada durante as execuções manuais pelo próprio pesquisador, observando legibilidade das métricas, distinção visual de severidade e associação entre alertas e cenários. Não houve aplicação de instrumento formal de coleta nem avaliação com desenvolvedores externos.
- **Comparação limitada com ferramentas de referência:** as leituras ADB serviram como apoio qualitativo. Em especial, `adb shell dumpsys gfxinfo` confirmou a superfície Flutter ativa, mas retornou contadores HWUI de frames zerados na execução em emulador, impossibilitando cálculo formal de erro de FPS em relação ao pacote.

5.4 Trabalhos Futuros

A continuidade deste trabalho pode avançar em cinco direções principais:

- **Validação quantitativa formal:** repetir cada cenário em múltiplas rodadas controladas, com amostra suficiente para calcular médias, desvio padrão, intervalos de confiança e erro percentual em relação a ferramentas de referência.
- **Comparação sistemática com ferramentas especializadas:** executar os mesmos cenários com Android Profiler, Perfetto e comandos ADB, registrando séries temporais comparáveis de FPS, memória, CPU e eventos de renderização.
- **Ampliação de dispositivos:** avaliar o pacote em diferentes perfis de hardware, incluindo aparelhos de entrada, intermediários e topo de linha, além de versões variadas do Android.
- **Suporte iOS mais completo:** evoluir os *platform channels* para métricas nativas de CPU e energia no ecossistema Apple, respeitando as restrições de *sandbox* da plataforma.
- **Avaliação com desenvolvedores Flutter:** conduzir estudo de usabilidade com participantes externos para medir clareza do painel, utilidade das recomendações e facilidade de integração do pacote em projetos reais.

Apêndice A

Evidências Visuais e Leituras ADB

Este apêndice reúne as capturas obtidas no aplicativo de exemplo do `collector_flutter` executado em modo *profile* no emulador Android `sdk gphone64 x86_64`, Android 16 (API 36), em 5 de junho de 2026. As telas foram capturadas por ADB e os valores complementares foram extraídos dos logs `EVIDENCE` emitidos pelo próprio aplicativo e dos comandos `adb shell dumpsys meminfo com.example.example` e `adb shell dumpsys gfxinfo com.example.example`.

Tabela A.1: Resumo das evidências coletadas no emulador Android.

Cenário	Sinais do <code>collector_flutter</code>	Leituras complementares por ADB
Reconstruções intensivas	30 <i>rebuids</i> ; FPS 55,4; P95 20,4 ms; 2 frames longos; CPU 22,1%; recomendação: “Tudo OK”.	<code>meminfo</code> : RSS total 199.208 KB e PSS total 113.674 KB. <code>gfxinfo</code> : processo e superfície Flutter ativos, porém contadores HWUI de frames retornaram 0.
Chamadas de rede	9 requisições; FPS 42,9; P95 139,5 ms; 4 frames longos; recomendação de performance de frames.	<code>meminfo</code> : RSS total 213.380 KB e PSS total 126.599 KB. <code>gfxinfo</code> : superfície Flutter ativa, mas sem contadores HWUI utilizáveis.
Alocação de memória	Painel registrou a execução do cenário, mas a leitura de memória do coletor apareceu como 0,0 MB no snapshot capturado.	<code>meminfo</code> durante a alocação: RSS total 236.480 KB e PSS total 222.232 KB, evidenciando aumento real de memória no processo.
Carga combinada	17 requisições; 54 <i>rebuids</i> ; FPS 44,6; P95 37,7 ms; CPU 75,3%; recomendações de performance de frames e uso moderado de CPU.	<code>gfxinfo</code> : superfície Flutter ativa, mas contadores HWUI zerados. Uma leitura posterior de <code>meminfo</code> registrou RSS total 128.988 KB e PSS total 107.293 KB após liberação parcial da carga.

Os contadores zerados de `gfxinfo` foram mantidos como evidência da limitação observada: nesta execução, o aplicativo Flutter renderizou por uma *SurfaceView*, e os agregados HWUI tradicionais não forneceram uma série de FPS comparável à métrica interna do pacote.



Figura A.1: Evidência visual do cenário de reconstruções intensivas de interface.



Figura A.2: Evidência visual do cenário de chamadas de rede simultâneas.



Figura A.3: Evidência visual do cenário de alocação intensiva de memória.



Figura A.4: Evidência visual do alerta de *jank* no painel.



Figura A.5: Evidência visual do cenário de carga combinada.

Apêndice B

Casos de Teste Automatizados

Este apêndice detalha os 60 casos de teste automatizados existentes na suíte do pacote `collector_flutter`. Em 5 de junho de 2026, o comando `flutter test`, executado no diretório `collector_flutter/`, concluiu com 60 testes aprovados e nenhuma falha.

B.1 Análise e Estatísticas

O arquivo `analyzer_test.dart` contém 24 casos:

1. `FrameStats.compute` retorna vazio para lista vazia.
2. Calcula média de frames corretamente.
3. Calcula P50 para lista ímpar.
4. Calcula P50 por interpolação para lista par.
5. Calcula P95 corretamente.
6. Calcula P99 corretamente.
7. Calcula desvio padrão corretamente.
8. Retorna desvio padrão zero para lista com todos os valores iguais.
9. Conta corretamente o número de amostras.
10. Calcula estatísticas para lista com um único elemento.
11. `AnalysisResult.empty` retorna zeros e sem problemas.
12. `hasIssues` retorna verdadeiro quando há problemas.
13. Converte memória em bytes para MB corretamente.
14. Garante que duas instâncias do analisador não compartilham estado de memória.
15. Calcula pico e tendência de memória com *timestamps* reais.

16. Calcula falhas, bytes e latência de rede.
17. Não gera alerta de memória por RSS alto isolado.
18. Gera alerta de memória apenas por tendência alta.
19. Não gera alerta de memória por tendência curta de inicialização.
20. Não gera alerta de rede com apenas uma requisição lenta e com falha.
21. Gera alerta de rede com 5 requisições e P95 alto.
22. Gera alerta de rede com 5 requisições e falhas.
23. Não gera *jank* com 15 frames e 5 frames longos.
24. Gera *jank* com 15 frames e 6 frames longos.

B.2 CPU e Bateria

O arquivo `cpu_battery_test.dart` contém 17 casos:

1. `CpuDataSource` retorna `-1.0` quando `MissingPluginException` é lançada.
2. `CpuDataSource` retorna valor do *platform channel* quando disponível.
3. `CpuDataSource` retorna `-1.0` quando `PlatformException` é lançada.
4. `BatteryDataSource.getBatteryLevel` retorna `-1` quando o plugin está indisponível.
5. `BatteryDataSource.getBatteryLevel` retorna o nível correto do *platform channel*.
6. `BatteryDataSource.isCharging` retorna falso quando o plugin está indisponível.
7. `BatteryDataSource.isCharging` retorna verdadeiro quando o dispositivo está carregando.
8. CPU elevada acima de 80% gera alerta.
9. CPU elevada no início da sessão não gera alerta.
10. CPU moderada entre 60% e 80% gera alerta.
11. CPU abaixo de 60% não gera alerta.
12. CPU indisponível, indicada por `-1`, não gera alerta.
13. Bateria baixa sem carregamento gera alerta.
14. Bateria baixa com carregamento não gera alerta.
15. Bateria acima de 20% não gera alerta.

16. `hasCpuData` retorna verdadeiro quando `cpuUsagePercent` ≥ 0 .
17. `hasBatteryData` retorna verdadeiro quando `batteryLevel` ≥ 0 .

B.3 Exportação

O arquivo `export_service_test.dart` contém 7 casos:

1. `ExportService.toJson` retorna uma *string* JSON válida.
2. Inclui a seção `session` com `sessionStart` e `rebuildCount`.
3. Inclui campos de frames com percentis.
4. Inclui seção de rede com lista de eventos.
5. Inclui recomendações serializadas com severidade como *string*.
6. `frameTimingsToCsv` retorna o cabeçalho CSV correto para modelo sem frames.
7. CSV vazio contém apenas o cabeçalho quando não há frames.

B.4 Recomendações

O arquivo `recommender_test.dart` contém 12 casos:

1. Retorna “Tudo OK” quando não há anomalias.
2. Retorna “Tudo OK” em estado ocioso com RSS alto e uma requisição lenta ou com falha.
3. Gera recomendação de performance quando FPS é menor que 40 em modo *debug/profile*.
4. Não gera recomendação de FPS quando FPS é igual a zero, indicando espera por dados.
5. Gera alerta de *jank* quando há pelo menos 6 frames longos.
6. Não gera alerta de *jank* quando há menos de 6 frames longos.
7. Não gera alerta de *jank* por P95 isolado sem problema identificado pelo analisador.
8. Gera alerta de rede quando há mais de 50 requisições.
9. Não gera alerta de rede quando há 50 requisições ou menos.
10. Não gera alerta de rede por P95 ou falha isolados sem problema identificado pelo analisador.
11. Gera múltiplas recomendações simultaneamente quando há múltiplos problemas.
12. Mantém os valores do enum de severidade em ordem crescente.

Referências Bibliográficas

- [1] PATHAK, A.; HU, Y. C.; ZHANG, M.; BAHL, P.; WANG, Y.-M. *Fine-grained Power Modeling for Smartphones Using System Call Tracing*. In: *Proceedings of the 6th European Conference on Computer Systems (EuroSys)*. ACM, 2011. p. 153–168. DOI: <https://doi.org/10.1145/1966445.1966460>. Acesso em: 20 abr. 2026.
- [2] HAO, S.; LI, D.; HALFOND, W. G. J.; GOVINDAN, R. *Estimating Mobile Application Energy Consumption Using Program Analysis*. In: *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013. p. 92–101. Disponível em: <https://2013.icse-conferences.org/content/estimating-mobile-application-energy-consumption-using-program-analysis.html>. Acesso em: 20 abr. 2026.
- [3] LI, D.; HAO, S.; GUI, J.; HALFOND, W. G. J. *An Empirical Study of the Energy Consumption of Android Applications*. In: *International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2014. p. 121–130. DOI: <https://doi.org/10.1109/ICSME.2014.34>. Acesso em: 20 abr. 2026.
- [4] RAVINDRANATH, L.; PADHYE, J.; AGARWAL, S.; MAHAJAN, R.; OBERMILLER, I.; SHAYANDEH, S. *AppInsight: Mobile App Performance Monitoring in the Wild*. In: *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2012. p. 107–120. Disponível em: <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/ravindranath>. Acesso em: 20 abr. 2026.
- [5] HOQUE, M. A.; SIEKKINEN, M.; KHAN, K. N.; XIAO, Y.; TARKOMA, S. *Modeling, Profiling, and Debugging the Energy Consumption of Mobile Devices*. *ACM Computing Surveys*, v. 48, n. 3, art. 39, 2015. DOI: <https://doi.org/10.1145/2840723>. Acesso em: 20 abr. 2026.
- [6] RUA, R.; SARAIVA, J. *A Large-scale Empirical Study on Mobile Performance: Energy, Run-time and Memory*. *Empirical Software Engineering*, v. 29, art. 31, 2024. DOI: <https://doi.org/10.1007/s10664-023-10391-y>. Acesso em: 20 abr. 2026.
- [7] SUN, Y.; FANG, J.; CHEN, Y.; LIU, Y.; CHEN, Z.; GUO, S.; CHEN, X.; TAN, Z. *Energy Inefficiency Diagnosis for Android Applications: A Literature Review*. *Frontiers of Computer Science*, v. 17, n. 1, art. 171201, 2023. DOI: <https://doi.org/10.1007/s11704-021-0532-4>. Acesso em: 20 abr. 2026.

- [8] BANGASH, A. A.; ENG, K.; JAMAL, Q.; ALI, K.; HINDLE, A. *Energy Consumption Estimation of API-usage in Mobile Apps via Static Analysis*. In: *20th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. IEEE/ACM, 2023. p. 272–283. DOI: <https://doi.org/10.1109/MSR59073.2023.00047>. Acesso em: 20 abr. 2026.
- [9] NAWROCKI, P.; WRONA, K.; MARCZAK, M.; SNIEZYNSKI, B. *A Comparison of Native and Cross-Platform Frameworks for Mobile Applications*. *Computer*, v. 54, n. 3, p. 18–27, 2021. DOI: <https://doi.org/10.1109/MC.2020.2983893>. Acesso em: 20 abr. 2026.
- [10] FLUTTER. **Flutter: Build Apps for Any Screen**. Disponível em: <https://flutter.dev>. Acesso em: 20 abr. 2026.
- [11] FLUTTER. **Flutter and Dart DevTools**. Disponível em: <https://docs.flutter.dev/tools/devtools/overview>. Acesso em: 20 abr. 2026.
- [12] ANDROID DEVELOPERS. **Profile Your App Performance**. Disponível em: <https://developer.android.com/studio/profile/android-profiler>. Acesso em: 20 abr. 2026.
- [13] PERFETTO. **Perfetto Tracing Documentation**. Disponível em: <https://perfetto.dev/docs/>. Acesso em: 20 abr. 2026.
- [14] APPLE. **Profiling Apps Using Instruments**. Disponível em: <https://developer.apple.com/tutorials/instruments>. Acesso em: 20 abr. 2026.
- [15] FIREBASE. **Firestore Performance Monitoring**. Disponível em: <https://firebase.google.com/docs/perf-mon>. Acesso em: 20 abr. 2026.
- [16] UNITAR; ITU. **Global E-waste Monitor 2024**. Geneva/Bonn: UNITAR; ITU, 2024. Disponível em: https://www.itu.int/hub/publication/d-gen-e_waste-01-2024/. Acesso em: 20 abr. 2026.
- [17] GIL, A. C. **Métodos e Técnicas de Pesquisa Social**. 7. ed. São Paulo: Atlas, 2019. Disponível em: <https://www.grupogen.com.br/metodos-e-tecnicas-de-pesquisa-social>. Acesso em: 20 abr. 2026.
- [18] LAKATOS, E. M.; MARCONI, M. A. **Fundamentos de Metodologia Científica**. 9. ed. São Paulo: Atlas, 2021. Disponível em: <https://www.grupogen.com.br/fundamentos-de-metodologia-cientifica/>. Acesso em: 20 abr. 2026.
- [19] YIN, R. K. **Estudo de Caso: Planejamento e Métodos**. 5. ed. Porto Alegre: Bookman, 2015. Disponível em: https://books.google.com/books/about/Estudo_de_Caso_5_Ed.html?id=EtOyBQAAQBAJ. Acesso em: 20 abr. 2026.
- [20] WOHLIN, C.; RUNESON, P.; HÖST, M.; OHLSSON, M. C.; REGNELL, B.; WESSLÉN, A. **Experimentation in Software Engineering**. Berlin: Springer, 2012. DOI: <https://doi.org/10.1007/978-3-642-29044-2>. Acesso em: 20 abr. 2026.

- [21] PRESSMAN, R. S.; MAXIM, B. R. **Software Engineering: A Practitioner's Approach**. 9. ed. New York: McGraw-Hill Education, 2020. Disponível em: <https://www.mheducation.com/highered/product/software-engineering-a-practitioners-approach-pressman.html>. Acesso em: 20 abr. 2026.
- [22] SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019. Disponível em: <https://www.bvirtual.com.br/NossoAcervo/Publicacao/engenharia-de-software-168127>. Acesso em: 20 abr. 2026.
- [23] MARTIN, R. C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Boston: Pearson, 2018. Disponível em: <https://www.informit.com/store/clean-architecture-a-craftsmans-guide-to-software-structure-9780134494166>. Acesso em: 20 abr. 2026.